Predicting Credit-Card Default: A Comparative Study of Machine-Learning Classifiers

RUAN Hangjing
G2607546F
hangjing001@e.ntu.edu.sg

LU Xinyu
G2603537D
xinyu057@e.ntu.edu.sg

LI Jingwen
G2607526K
jingwen013@e.ntu.edu.sg

Abstract

Banks use default models as cut-offs, so we judge ranking and operating point. On the UCI credit-card data (29,965 cardholders, 22.1% defaulters, drawn as the 2005 card-debt crisis began) we compare ten classifiers, choosing everything by training cross-validation (CV) and freezing each configuration at its first test scoring. XGBoost, gradient boosting and the random forest tie (CV AUC 0.785–0.786; XGBoost test 0.789), 0.025–0.026 above the logit. Post-freeze training-CV checks show where the signal lies: adding September's record to limit and demographics lifts XGBoost's AUC from 0.63 to 0.77; a third of the logit's gap is reading repayment codes as evenly spaced numbers, most of the rest one-variable curves; interactions are small; demographics add 0.002 CV AUC but raise fairness issues. Resampling before the split fakes a 0.914 AUC. At 5:1, moving the logit's cut-off saves 1,937 test cost units; switching to XGBoost saves 145 more $[-18, 299]$.

Contribution statement. Group B19. Model families are owned end to end (code, tuning, interpretation, presentation): RUAN Hangjing the KNN benchmark, the logistic family, nested cross-validation and PCA/K-means; LU Xinyu the pruned tree, random forest and bagging, gradient boosting and XGBoost, TreeSHAP, monotone constraints and partial dependence; LI Jingwen the support-vector machines, the neural network, LDA/QDA/naive Bayes, the class-imbalance study and leakage demonstration, Platt recalibration, and the freeze and aggregation code. Shares per section (RUAN/LU/LI, %): §1 30/20/50; §2 25/15/60; §3 30/25/45; §4 30/35/35; §5 15/60/25; §6 40/15/45; §7 34/33/33; code and reproducibility 20/25/55; presentation 33/33/34 (Table A4). Overall, the unweighted mean of these nine shares rounded: RUAN 29%, LU 29%, LI 42%, agreed by all three members. Not shared with any other module.

1 Introduction

A default model in a crisis vintage. Every probability of default (PD) here belongs to a crisis vintage. The data come from Taiwan's 2005–06 card-debt crisis [19]: the central bank dates the first repayment problems to September 2005, our last feature month [4]. Revolving balances peaked at NT$495.3 bn in November 2005 and fell 48.8% by December 2008 [9]. Our 22.1% default rate is consistent with a stressed book or an undocumented, defaulter-heavy sample (§7, iii). Errors cost unequally: a missed defaulter's balance is lost; a wrongly flagged good client risks a thin margin.

Why explanations matter. Lenders are now expected to explain a credit score, so we pair accuracy with Shapley additive explanations (SHAP), monotone constraints and a logistic scorecard. The EU AI Act classes creditworthiness scoring of individuals as high-risk (Annex III, 5(b)) [8]. Singapore's FEAT principles ask that customers can learn which data were used and how they affected a decision (Principle 13) [15]; these are present-day benchmarks, and we claim no compliance. In the

2024 Bank of England–FCA survey, gradient boosting was the most common model type (32% of use cases) and SHAP (64%) among the most popular explainability methods [1].

Design and objectives. The study is comparative [12]: a K-nearest-neighbour (KNN) benchmark and a logistic scorecard are the references, each other family is explained by how it learns, and TreeSHAP [14], monotone XGBoost [6] and partial dependence [10] go beyond the syllabus. We ask (1) *complexity*: what flexible models add beyond noise; (2) *operating point*: whether the cut-off matters more than the model; (3) *imbalance*: what weighting and resampling do inside the folds and before the split; (4) *inference*: which characteristics predict default, in which direction, what personal attributes add at what fairness cost, and whether the importance measures agree.

2 Data

Source, scope and what the label means. The data are 30,000 existing cardholders of one Taiwanese bank (*Default of Credit Card Clients*, UCI Machine Learning Repository, `https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients`, doi:10.24432/C55S3H, CC BY 4.0 [18]; described in [19]). We predict default on the October 2005 payment from April–September 2005 histories, so a flag triggers an account-management action on a current client (a call, a reminder, a limit review), not an approval. The monthly variables precede the label; the limit and demographics are undated snapshots, assumed recorded by September 2005 (if the bank cut limits after missed payments, the limit would partly record the outcome). Removing 35 rows that repeat another client's record exactly, label included (mostly dormant profiles), leaves 29,965 clients, 29 predictors (Table A1) and 22.1% defaulters: "never default" is 77.9% accurate yet catches nobody. Code: `github.com/jingwen013-design/mh6805-credit-default` (`code/run_all.sh` regenerates every number).

Repayment codes are states, not a scale. PAY_1–PAY_6 record each month's status: -2 no consumption, -1 paid in full, 0 revolving credit, $k \geq 1 = k$ months late (UCI documents only -1 and 1–9; the readings of -2 and 0 are widely used, not documented). Non-delinquent codes have no order: September default is 13.7% at -2, 16.9% at -1 and 12.9% at 0 (training rows). Code 1 also changes meaning: it appears 2,540 times in September but 21 times in the five earlier months, mixing dormant accounts with clients recovering from two months late (Table A21), so compare codes within a month. A scorecard should weight these states separately (§4).

Where the risk gradients are. Default rates change steeply with recent arrears and the limit, not with bill size. September default climbs from 12.9% at revolving credit to 67.6% at two months late, two payments already missed (code 1 is the mixture above). It falls steadily as the limit rises, but the bank sets the limit: a high limit marks a client it already judged safe, so raising a limit would not make anyone safer. Bill size alone ranks no better than chance (univariate area under the ROC curve, AUC, 0.50–0.52). Utilisation (bill over limit) is U-shaped: default is lowest for light users and highest near the limit, where no buffer is left. It is also above average with no positive balance, but most such clients carry the odd September code 1, so that arm is of uncertain meaning. Sex and age move the rate by at most 8 points (training rows; Table A2, Figures A7–A9).

What happens before the split. Three fixed steps estimate nothing, so they precede the split; everything estimated (scaling, one-hot levels, resampling, thresholds) is re-fitted inside each fold's `Pipeline` (Table A3). (i) An a-priori mapping of EDUCATION and MARRIAGE codes the UCI dictionary does not list ({0,5,6} and 0 to "others"), so the "others" education bucket (343 training clients, 6.7% default) is a coding artefact. (ii) Six engineered features: two utilisation ratios, a payment ratio capped at 5, months in arrears, worst delay and balance trend (the delay counters count -2, -1 and 0 as 0). (iii) Removal of the 35 repeated rows. Filling the payment ratio with 0 when all six bills are non-positive makes owing nothing look like paying nothing. The fill is not neutral: these 633 training clients default at 37.0% against 21.7%, so a bank must document it.

3 Methods

Protocol, and a record of every test read. Every choice is made on training rows; the test set is a report card read more than once. One stratified split gives 20,975 training and 8,990 test clients

(seed 2607526). Five shared stratified cross-validation (CV) folds give each training client an out-of-fold (OOF) probability from a model that never saw it. Hyper-parameters are chosen by CV AUC, and both cut-offs (maximum F_1; minimum 5:1 cost) on these OOF probabilities. Round-1 configurations were scored on 16–18 September and entered in the freeze register on 19 October (46 entries, Table A9); since then `common.evaluate()` refuses to re-score a registered configuration that has changed. Before that, the dated test log is the guard: besides identical re-runs, only RUAN Hangjing's 30 September re-fits of the plain logit and KNN (same settings) were re-scored. Each configuration thus counts as frozen at its first scoring. The log (`results/TEST_SET_LOG.md`, Table A10) also dates eight diagnostic fits outside the register. None set a model, feature or threshold; the linear support-vector machine (SVM) moved to squared-hinge loss after one such read, but training CV backs the move. *Post-freeze checks* are the 21 October analyses: training five-fold CV, never test-scored, selecting nothing (`results/POST_REVIEW_CHECKS.md`; Tables A21–A33).

Tuning budgets, nested CV, and how to read the numbers. Tuning is like-for-like only for a core six (logit, KNN, pruned tree, forest, XGBoost and multilayer perceptron, MLP). Gradient boosting was tuned on half the training rows, the SVMs on 6,000 rows, and bagging's leaf size and the monotone vector were fixed a priori (Tables A5–A7). The grids are small: 9 settings for the forest to 84 for the pruned tree, with two or three values for most parameters; widening them at the edges gains at most 0.0003 for XGBoost and 0.0008 for gradient boosting, and only the forest's leaf size is left open (Table A24). Nested CV re-tunes inside each outer fold, so no score grades its own tuning; for KNN and the logistic family it leaves the frozen choices standing (KNN 0.768 ± 0.008 against 0.770; the plain logit kept; Table A11). On all training rows (training CV, post-freeze), gradient boosting's grid ranks the frozen setting third, only 0.0008 behind (Table A24); the ensembles were not nested (§7, iv). *Reading the numbers*: AUC is the chance that a random defaulter scores above a random payer (0.5: chance). Average precision (AP) is the mean precision down the ranked list (no skill: 0.221, the default rate). Brier is the mean squared error of the PD. Cost at r:1 is $r\times$ missed defaulters + false alarms, so one unit is one false alarm; a cross-fitted cost sets each fold's cut-off on the other four. "$+0.0082$ [$+0.0021$, $+0.0143$], 5/5" is the mean of five fold-paired differences, its 95% interval, widened because the folds share training rows (Nadeau–Bengio [16]), and the count of positive folds. R refits on the same folds match 13 of 15 Python CV AUCs within 0.005, so the ranking is not a software artefact (Table A33).

4 Results

Four tiers, and a tie at the top. The four tiers of Table 1 hold in every fold (Figure A1). The forest averages 500 de-correlated trees on bootstrap samples [3]; boosting adds small trees in turn, each fitting the errors left [6]. Both learn steps and curves unaided and tie: XGBoost leads the other two by 0.0013–0.0014 training-CV AUC, both intervals including zero (Table A23). A bank may pick any on cost or explainability; XGBoost leads only by the preset highest-CV rule.

Local and step-wise models: KNN and the tree. The KNN benchmark and the pruned tree tie because both, in effect, group clients by their repayment codes. KNN scores a client by the default share among its 101 nearest neighbours in 32 columns. As the status codes are discrete, a neighbourhood is in effect "clients with the same recent record", and compressing the columns to 15 principal components does not help (training CV, post-freeze; Table A24). The pruned tree, 17 rules splitting first on the worst delay, ties KNN on CV (0.769 against 0.770, ahead in 3 of 5 folds); its higher test AUC is reported, not used to rank. An auditable tree matches the benchmark.

Why the logit trails: coding first, shape second. The logit reads each status code as one evenly spaced number, forcing $-2 < -1 < 0 < 1 < \ldots$ onto a line. Coded by meaning (months late plus flags for -2 and -1), "no consumption" and "paid in full" get their own risk. This lifts the logit from 0.760 to 0.768 ($+0.0082$ [$+0.0021$, $+0.0143$], 5/5), within noise of KNN; KNN and the MLP, which already let each code act alone, gain nothing (training CV, post-freeze; Table A22). Recoding closes most of the gap to KNN but only a third of the gap to XGBoost: even a one-hot logit, free to give each code any risk, still trails XGBoost by 0.017 (5/5 folds). Bills and utilisation move together, so their coefficients take opposite signs: read them as a block (Figure A29). For a bank, recoding cheaply lifts a transparent scorecard to the benchmark.

Table 1: Headline model per family by training CV AUC (mean of five fold AUCs ± sample SD; the selection score; ranks among 11; lasso logit not shown). Test columns ($n = 8{,}990$; frozen, ranking nothing): point estimates, 95% paired-bootstrap intervals ($B = 500$); t: 5:1 cut-off (training OOF). †Mean of shared-fold OOF AUCs (pooled 0.761; 0.768 on its tuning subsample).

Model	CV AUC ± SD	Ranks	Test AUC [95% CI]	Test AP	Brier	t	Cost at t
Tree ensembles — tied (ranks 1–3 in every fold)							
XGBoost [6]	0.786 ± 0.008	1–2	0.789 [0.776, 0.800]	0.578	0.131	0.15	4,981
Gradient boosting	0.785 ± 0.008	2–3	0.787 [0.775, 0.798]	0.576	0.131	0.14	5,060
Random forest, 500 trees	0.785 ± 0.009	1–3	0.787 [0.774, 0.798]	0.576	0.132	0.17	4,998
Neural network (4th in every fold)							
MLP (64–32)	0.776 ± 0.008	4	0.773 [0.760, 0.785]	0.551	0.135	0.19	5,136
Benchmark tier (ranks 5–7)							
KNN ($K = 101$), benchmark	0.770 ± 0.010	5–7	0.771 [0.758, 0.782]	0.542	0.136	0.16	5,219
Pruned tree (17 leaves)	0.769 ± 0.008	5–6	0.776 [0.763, 0.787]	0.535	0.133	0.16	5,085
SVM (RBF)	0.764† ± 0.013	6–7	0.762 [0.749, 0.774]	0.494	0.145	0.12	5,155
Linear and generative (ranks 8–11)							
Logistic, baseline	0.760 ± 0.011	8–9	0.764 [0.751, 0.776]	0.523	0.138	0.15	5,126
LDA	0.757 ± 0.012	10	0.759 [0.746, 0.772]	0.525	0.139	0.12	5,181
Gaussian naive Bayes	0.736 ± 0.012	11	0.740 [0.726, 0.753]	0.504	0.203	0.25	5,337

Smooth and generative models: close in ranking, not in probabilities. Smooth models add little ranking; wrong assumptions about the data distort the PDs. The MLP is fourth in every fold, $+0.006$ over KNN in all five: suggestive, not decisive (one seed; CV AUC chose its grid). The radial-basis-function (RBF) SVM maximises a margin, so its scores become probabilities only after Platt scaling, a logistic map from score to PD fitted on held-out scores [17]. Linear discriminant analysis (LDA) assumes bell-shaped amounts with one shared covariance, though bills are heavily skewed, and naive Bayes counts one balance six times, as the bills are highly correlated. Their training OOF calibration slopes (post-freeze; 1 = calibrated, lower = PDs too extreme) are 0.749 and 0.141 against 1.015 for XGBoost (Table A23, Figure A16): such PDs must not price or provision.

Class imbalance: weight the decision, not the data. Class weighting tells the logit there are more defaulters than exist. It mostly shifts the intercept ($+1.24$ log-odds), which moves the cut-off and spoils calibration until Platt scaling undoes it (Table A16). Run inside the folds with fixed, untuned learners, SMOTE (synthetic defaulters interpolated between real neighbours [5]) costs the forest 0.0055 training-CV AUC, while weighting and under-sampling move it by under 0.001 (Figure A33). Applied after one-hot encoding, SMOTE also invents impossible clients, with fractional dummies and status codes. Run before the split, SMOTE lifts the forest's "test" AUC to 0.914 against an honest 0.774, as synthetic "test" defaulters interpolate training ones. Set the cut-off from costs or recalibrate; never resample before splitting.

Segmentation stays descriptive. K-means on 10 principal components picks five segments, but they overlap (silhouette 0.27: weak to moderate separation) and shift across random starts (Table A17). A segment label is a coarse summary of columns the logit already sees, so in training CV it moves the logit's AUC by $+0.0001$ [-0.0004, $+0.0007$]. Not adopted, it was never test-scored (a round-1 $K = 4$ version was; Table A12). Segment default rates differ largely because the inputs include delay and limit: segments describe the book; they do not price risk (Table A18).

5 Interpretation

Why repayment status dominates. Next month's default is largely the next step of a delinquency process, so the latest status is nearly a lagged copy of the label. Arrears persist or deepen: 59.1% of clients two months late in September, and 85.9% of those three months late, were two months late in August (training rows; Table A21). Repayment is also the only liquidity signal the bank sees (income and debts are unseen). Adding September's status, bill and payment to limit and demographics lifts XGBoost's CV AUC from 0.63 to 0.77. Older months add 0.013 as raw months, or 0.017 through six-month summaries such as months in arrears, which tell a new slip from a chronic one (5/5 folds

each; training CV, post-freeze; Table A26, Figure A2). A crisis-vintage model thus learns crisis roll rates (how fast arrears deepen) and may overstate them in calmer years (§7, iii).

Where the ensembles' edge comes from: shape, then small interactions. Most of XGBoost's edge over the logit is one-variable curves; interactions add little. A depth-1 booster (one split per tree) adds one-variable effects of any shape; depth 3 lets variables interact (trees re-tuned per depth). The additive booster recovers 0.0225 of the 0.0261 gap (5/5 folds) and depth 3 adds only 0.0036, an upper estimate (selected on the same folds; training CV, post-freeze; Table A25; same split on raw columns). The one-hot logit gives each status code its own risk and still trails, so the remaining shape likely sits in other columns, such as utilisation's U in partial-dependence plots (PDPs [10]); we did not measure which. Friedman's H (the share of a pair's effect beyond the two separate effects) is 0.061 for status $\times$ limit: little support for strong interactions in the pairs tested (Figures A26, A27). A binned scorecard or additive model should thus recover most of the gain and stay explainable.

Which variables matter, and in which direction. All four measures put September status first, then disagree for known reasons. The forest's impurity importance (MDI) credits every training-row split. Each split sees only a random subset of columns, so when September status is not on offer, correlated delay columns stand in and collect its credit. Permutation, shuffling a column on rows a tree did not see (out of bag) [3], shows how much *this fitted model* leans on it. There the August and July codes, 4th and 5th on MDI, rank only 22nd and 24th. Refitting without a column measures what is lost: of six columns refitted, only September status costs over 0.001 out-of-bag AUC, and the delay counters recover most of even that (Table A20). Across models, SHAP (XGBoost) and permutation (forest) agree (rank correlation 0.77), MDI and permutation far less (Table A19); September status, worst delay and months in arrears carry one signal, so read them as a block. A Shapley value [14] splits a client's score into feature contributions summing to its log-odds minus the average: arrears push towards default and a larger limit away (Figure A22), an association (§2). These are reasons a bank can give a customer [15]; they describe the model, not the world.

Monotone constraints: impose only what credit logic guarantees. A monotone constraint fixes a direction in advance: all else equal, the score never falls as a client falls further behind. The frozen version also forced it up from -2 to -1 to 0 and with utilisation and balance trend (Table A8). §2 contradicts both the order and the rise, so this costs 0.0019 CV AUC (no fold better). Credit logic is certain only from revolving credit upwards. Up to three months late the free booster already obeys: moving 200 training clients from revolving credit to one, two and three months late (counters updated) never lowers a score. Its wrong-way steps lie among non-delinquent codes or in the thin 4- and 5-month cells (57 and 16 clients), where observed default also falls. Constraining only delay (score up) and limit (score down) is within 0.0005 of the free booster (training CV, post-freeze; Table A29). A bank can thus promise, almost for free, that slipping from revolving credit into arrears never lowers a score (rare jumps from -2 or -1 into arrears still lower it slightly).

Personal attributes: little ranking power, real fairness trade-offs. Sex, age, marriage and education barely help ranking: removing them costs XGBoost 0.002 CV AUC (no fold better without; training CV, post-freeze; Table A27). Yet its cross-fitted 5:1 cost rises in all five folds, as age carries information the limit does not. When groups default at different rates, no score can be both calibrated within each group and equal in its error rates [7, 13]. Men default 1.19 times as often as women. XGBoost's PDs match each large group's observed rate, so it flags men 1.18 times as often, partly through SEX and MARRIAGE; men who would repay are therefore flagged more often too (training OOF, post-freeze; Table A28). The logit flags men more often still, as one SEX coefficient shifts every man's log-odds alike and more of its scores sit near the cut-off. Dropping the columns narrows the sex gap but under-predicts men, flags more under-30s (without age, their low limits make them look riskier) and leaves proxies, mainly the limit (Figure A3). "Fairness through unawareness" is thus a trade-off a bank must choose and document, not a fix; present-day frameworks class such scoring as high-risk [8] and want attributes justified and reviewed [15].

6 Choosing the operating point

Why 0.5 is the wrong cut-off, and 5:1 a scenario, not a fact. A cut-off of 0.5 is optimal only when both errors cost the same. With 22% defaulters few clients reach a PD of 0.5: on the test set

Table 2: How the decision moves with the cost ratio r. Left (training OOF, frozen models): cheapest model (cut-off), top-3 cost spread, saving against the best blanket rule (flag nobody for $r \leq 3$, all for $r \geq 5$). Right (test set, frozen models; 95% paired bootstrap, $B = 1{,}000$): threshold saving = logit at 0.5 minus at its training-OOF cut-off $t_{\log}$; model saving = XGBoost's further reduction at its own cut-off. [a]GBM (gradient boosting) if cross-fitted. [b]Flags 96% (cheapest) and 99.9% (logit). Fold intervals of the cheapest's lead (all include zero), all ratios: Tables A15, A31.

		Training OOF (selects nothing)			Test set, frozen models		
r	$1/(1+r)$	Cheapest (cut-off)	Top-3 spread	vs blanket	$t_{\log}$	Threshold saving	Model saving
1	0.500	GBM (0.49)	0.69%	18.9%	0.50	0	87 [44, 131]
2	0.333	Forest (0.33)	0.98%	31.7%	0.33	238 [141, 338]	123 [65, 185]
3	0.250	XGBoost (0.27)[a]	0.80%	40.1%	0.30	684 [541, 830]	127 [47, 214]
5	0.167	XGBoost (0.15)	1.03%	30.8%	0.15	1,937 [1,604, 2,218]	145 [−18, 299]
10	0.091	Forest (0.09)	0.90%	12.7%	0.10	7,121 [6,395, 7,791]	420 [187, 646]
20	0.048	KNN (0.03)[b]	0.64%	2.5%	0.01[b]	20,335 [18,909, 21,621]	150 [−52, 333]

the frozen logit at 0.5 catches only 32% of defaulters and, at 5:1, costs more than flagging everyone (Table A13). The 5:1 ratio is borrowed from the German credit data (Statlog cost matrix [11]), not measured for this issuer. Our rough cost anchors point higher. If a missed defaulter loses 50–70% of the median defaulter's bill, and a false alarm forgoes one to three months of a 1.5% monthly margin on the median good client's bill, the ratio is 9.6:1 to 40.5:1. That is an upper bound, as some defaulters later pay (Table A14). A bank should read its own ratio off the 1:1–20:1 sweep (Table 2) and set the cut-off on training OOF scores.

The cut-off, not the ensemble, is the lever. Moving the cut-off saves far more than changing the model. At r:1 one caught defaulter is worth r false alarms, so the cheapest cut-off is where only one in $r + 1$ of the next clients flagged would default. For a calibrated model that is a PD of $1/(1 + r)$, 0.167 at 5:1, close to the searched 0.15 for the logit and XGBoost (badly calibrated naive Bayes needs a higher one; training OOF; Figures A4, A14). At 5:1 on the test set, moving the logit's cut-off saves 1,937 cost units (false-alarm equivalents); switching to XGBoost saves a further 145, but its interval includes zero. At 10:1, nearer our anchors, both savings are clearly positive (Table 2). The 5:1 flag is broad: XGBoost flags about half the test clients, 1.8 false alarms per defaulter caught (Table A13), so it suits only a cheap action such as a reminder.

The best ranker need not be the cheapest. AUC averages over all cut-offs, while a bank works at one, where ROC curves can cross: KNN, with a higher AUC than the logit, costs more at 5:1 on test (5,219 against 5,126; Table 1). Among the tied ensembles the cheapest rotates as the ratio moves from 1:1 to 15:1 (training OOF; Table A31), so we name no cost winner. Choose the model for ranking, the cut-off from the bank's costs, and calibrate; above about 15:1 most are flagged, so a model's value lies in ordering a graded action (reminders for most, calls for the riskiest).

A one-line rule, and who is on the first call list. "Flag anyone with any payment delay in six months" has no fitted parameter yet gets most of the way to the models' cost; it misses the 35.5% of defaulters never late, where balances, limit and utilisation earn their place. At 5:1 in training CV XGBoost costs 7.7% less than the rule (5/5 folds), the logit almost nothing (post-freeze; Table A30; the rule was the cheapest of four candidates on these rows, a mild selection effect). A team that can call only the riskiest 10% gains little from a model. On training OOF scores, XGBoost's top 10% catches 31.5% of defaulters and September status alone 30.9%, because the top of any ranking is filled with clients already two or more months late (Table A32). A model's value over the one-line rule is real but modest and lies further down the list: size the action to capacity.

7 Conclusion and limitations

Answers. (1) *Complexity*: the three ensembles tie, 0.025–0.026 CV AUC above the logit (XGBoost on test: $+0.0250$ [$+0.0183$, $+0.0314$]); a third of the gap is status coding, most of the rest one-variable shape, interactions small (training CV). (2) *Operating point*: from 2:1 to 20:1 the threshold saving exceeds the model saving: the cut-off, not the model, is the lever. (3) *Imbalance*:

in-fold weighting and under-sampling move the intercept, not the ranking, and SMOTE costs the forest 0.0055 CV AUC; resampling before splitting fakes a 0.914 test AUC. (4) *Inference*: recent status dominates, as default continues a delinquency process; demographics add 0.002 training-CV AUC at a fairness cost; constraining only the arrears direction is almost free.

Limitations. (i) *Retrospective test use* (§3): until 19 October test AUC chose the bootstrap's reference model and the four models in the figures, and nine Platt variants were scored on 19 October after a 6 October review had applied their maps to test scores (Platt was pre-committed on 30 September); the test set is a dated report card, not a fresh holdout. (ii) *Illustrative cost ratio* (§6). (iii) *Vintage and drift*: one Taiwanese bank at the onset of a card-debt crisis; base rates move several-fold over a cycle (US card delinquency 6.77% in 2009Q2, 1.53% in 2021Q3 [2], an analogy), so reuse needs PDs re-based to the new default rate, a re-estimated cost ratio and out-of-time validation. (iv) *Design*: a cross-section and one split; the ensembles were not nested, so their internal order cannot be separated from selection optimism; some grids end where CV might still rise (Table A24; clearly so only at the forest's largest leaf size, 20); single seeds for the MLP, forest and XGBoost. (v) *Attributes and causality*: fairness numbers are a training-OOF diagnostic, not a compliance assessment, and SHAP, PDPs and coefficients describe fitted models, not causes. *Future work*: an out-of-time holdout, bureau and income data, measured losses, an additive scorecard and nested multi-seed ensembles.

References

Web sources accessed 21 October 2026.

[1] Bank of England & Financial Conduct Authority (2024) *Artificial intelligence in UK financial services – 2024*, 21 Nov. `https://www.bankofengland.co.uk/report/2024/artificial-intelligence-in-uk-financial-services-2024`

[2] Board of Governors of the Federal Reserve System (2026) *Charge-off and delinquency rates on loans and leases at commercial banks* (credit cards, all banks, seasonally adjusted). `https://www.federalreserve.gov/releases/chargeoff/delallsa.htm`

[3] Breiman, L. (2001) Random forests. *Machine Learning* **45**(1):5–32.

[4] Central Bank of the Republic of China (Taiwan) (2008) *Financial Stability Report*, June 2008, Box 2. `https://www.cbc.gov.tw/en/public/Attachment/931716211071.pdf`

[5] Chawla, N. V., Bowyer, K. W., Hall, L. O. & Kegelmeyer, W. P. (2002) SMOTE: synthetic minority oversampling technique. *J. Artificial Intelligence Research* **16**:321–357.

[6] Chen, T. & Guestrin, C. (2016) XGBoost: a scalable tree boosting system. *Proc. KDD*, pp. 785–794.

[7] Chouldechova, A. (2017) Fair prediction with disparate impact: a study of bias in recidivism prediction instruments. *Big Data* **5**(2):153–163.

[8] European Union (2024) Regulation (EU) 2024/1689 (Artificial Intelligence Act), Annex III, point 5(b). *Official Journal* L, 12.7.2024. `https://eur-lex.europa.eu/eli/reg/2024/1689/oj`

[9] Financial Supervisory Commission (Taiwan), Banking Bureau, *Credit-card business statistics*, monthly time series (index-7.xlsx), Open Government Data Licence v1.0. `https://www.banking.gov.tw/webdowndoc?file=/stat/cc/index-7.xlsx`

[10] Friedman, J. H. (2001) Greedy function approximation: a gradient boosting machine. *Annals of Statistics* **29**(5):1189–1232.

[11] Hofmann, H. (1994) *Statlog (German Credit Data)*. UCI Machine Learning Repository, doi:10.24432/C5NC77.

[12] James, G., Witten, D., Hastie, T. & Tibshirani, R. (2021) *An Introduction to Statistical Learning*. Springer.

[13] Kleinberg, J., Mullainathan, S. & Raghavan, M. (2017) Inherent trade-offs in the fair determination of risk scores. *Proc. ITCS*.

[14] Lundberg, S. M. & Lee, S.-I. (2017) A unified approach to interpreting model predictions. *Advances in NeurIPS* **30**:4765–4774.

[15] Monetary Authority of Singapore (2018) *Principles to promote fairness, ethics, accountability and transparency (FEAT) in the use of artificial intelligence and data analytics in Singapore's financial sector*. `https://www.mas.gov.sg/publications/monographs-or-information-paper/2018/feat`

[16] Nadeau, C. & Bengio, Y. (2003) Inference for the generalization error. *Machine Learning* **52**(3):239–281.

[17] Platt, J. C. (1999) Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, MIT Press, pp. 61–74.

[18] Yeh, I.-C. (2009) *Default of Credit Card Clients* [Dataset]. UCI Machine Learning Repository, doi:10.24432/C55S3H.

[19] Yeh, I.-C. & Lien, C.-H. (2009) The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Syst. Appl.* **36**(2):2473–2480, doi:10.1016/j.eswa.2007.12.020.

A Appendix

Everything in this appendix is generated from `results/` by `code/make_report_appendix.py`, so the tables cannot drift from the artefacts submitted with the code. The course guideline exempts appendices from the seven-page limit on the body. Section A.1 documents the data, the protocol and the frozen models; Section A.2 collects post-freeze checks on the training rows, with their figures in Section A.3; `results/TEST_SET_LOG.md` dates every read of the test rows (Table A10); the figures of the data and the frozen models follow from Section A.4.

A.1 Data, protocol and frozen models

Table A1: Variables in the cleaned data set: 29 predictors and the target, 29,965 clients after removing 35 rows identical to another row on every column except the ID (the key included the label). The monthly variables are dated; the credit limit and the demographics are undated snapshots.

Group	Variables	Type	Meaning
Credit line	`LIMIT_BAL`	numeric	credit limit granted (NT$), individual plus family
Demographics	`SEX`, `EDUCATION`, `MARRIAGE`, `AGE`	3 categorical, 1 numeric	gender; graduate / university / high school / others; married / single / others; age in years. EDUCATION and MARRIAGE: an a-priori mapping of codes the UCI dictionary does not list ($\{0, 5, 6\} \to$ others; $0 \to$ others)
Repayment status	`PAY_1`, ..., `PAY_6`	ordinal ($-2 \ldots 8$)	monthly status Sep $\to$ Apr 2005: -2 no consumption†, -1 paid in full, 0 revolving credit†, $k \geq 1$ = k months late; †not defined in the UCI documentation (a widely used reading)
Bill statements	`BILL_AMT1`, ..., `BILL_AMT6`	numeric	monthly bill amount Sep $\to$ Apr (NT$); may be negative
Payments	`PAY_AMT1`, ..., `PAY_AMT6`	numeric	amount paid in each month Sep $\to$ Apr (NT$)
Engineered	`UTIL_1`, `UTIL_MEAN`, `PAY_RATIO_MEAN`, `N_MONTHS_DELAYED`, `MAX_DELAY`, `BILL_TREND`	numeric	utilisation = bill over limit (September; six-month mean); `PAY_RATIO_MEAN` = mean over months of the payment in month k over the bill of the same month k (bills ≤ 0 skipped), capped at 5, set to 0 when all six bills are non-positive (a payment usually settles the previous statement); months with status ≥ 1; worst status, floored at 0; (September bill $-$ April bill) over the limit
Target	`DEFAULT`	binary	1 = default on the October 2005 payment (22.1% in training)

Table A2: Default rate along the two strongest credit-risk gradients, training partition only ($n = 20{,}975$; overall rate 22.1%). Source: `results/eda_summary.json`. A status is compared within a month, not across months: code 1 (one month late) is recorded almost only in September and mixes clients with no positive September bill and clients rolling back from two months late (Table A21).

September status	Clients	Default	Credit-limit quintile	Clients	Default
No consumption (-2)	1,904	13.7%	NT$10k–50k	5,429	32.1%
Paid in full (-1)	3,941	16.9%	NT$50k–100k	3,370	25.5%
Revolving credit (0)	10,380	12.9%	NT$100k–180k	4,224	19.5%
1 month late	2,540	33.9%	NT$180k–270k	3,810	17.1%
2 months late	1,871	67.6%	NT$270k–1M	4,142	13.7%
3 months late	234	77.4%			

Table A3: Data-quality checks run on the raw download before modelling, and what was done about each finding. Reproduced verbatim from `results/eda_data_quality.csv`; the three cleaning steps that precede the split (duplicate removal, the fixed recoding of undocumented codes, the row-wise engineered features) estimate nothing from the data, and everything else is left to the in-fold pipeline. The counts are over the full file (30,000 raw or 29,965 cleaned rows, test partition included); labels are used only for the class balance, the split check and the default rates marked as training-partition.

Check	Finding	Action
Missing values	0 cells (all 24 columns complete)	None needed for the 24 raw columns; the engineered PAY_RATIO_MEAN is undefined for the 936 cleaned rows whose six bills are all non-positive and is set to 0 (row: Zero bills in all six months)
Exact duplicate rows	35 of 30,000 (0.12%)	Dropped in common.clean() → 29,965 rows
Data types	All 24 columns integer; no strings, no dates	—
EDUCATION undocumented codes 0/5/6	0: 14, 5: 280, 6: 51 (total 345, 1.15%)	Recoded to 4 = others/unknown
MARRIAGE undocumented code 0	54 rows (0.18%)	Recoded to 3 = others
PAY_1..6 undocumented codes -2 and 0	PAY_1: -2 in 2,759 rows, 0 in 14,737 rows (codebook lists only -1 and 1..9)	Kept as ordinal: -2 = no consumption, -1 = paid in full, 0 = revolving credit / minimum paid, k>0 = k months late
Negative bill amounts	3,932 cells in 1,930 rows (6.4%); min BILL_AMT = -339,603	Kept — a negative bill is a credit balance (overpayment / refund)
Zero payments	PAY_AMT = 0 in at least one month for 14,542 rows (48.5%); all six months zero for 1,432 rows (4.8%)	Kept — 'no payment' is informative; PAY_RATIO_MEAN handles zero bills
Zero bills in all six months	866 of the raw 30,000 rows (2.9%); 936 of the 29,965 cleaned rows (3.12%) have all six bills NON-POSITIVE	Kept — inactive or credit-balance accounts. For the non-positive group PAY_RATIO_MEAN is undefined and imputed 0; in the training partition that group defaults at 37.0% against 21.7% for the rest, so the imputed value is informative rather than neutral
Bill above credit limit (UTIL_1 > 1)	2,115 rows (7.1%); max UTIL_1 = 6.46	Kept — over-limit usage is a genuine risk signal
LIMIT_BAL range	10,000 – 1,000,000 NT$ (median 140,000)	Plausible; heavy right tail → StandardScaler inside the pipeline, no trimming
AGE range	21 – 79 years (median 34)	Plausible; no outlier removal
Class imbalance	6,630 defaults of 29,965 = 22.1%	Kept as is; addressed downstream by class_weight / threshold tuning; AUC & AP as metrics
Train / test split	20,975 / 8,990 rows, stratified 70/30, seed 2607526; default rate 22.13% / 22.12%	Fixed once in data/processed. Scope of this table: counts are over the full download, test rows included (feature counts behind fixed rules; nothing is estimated from them); test labels enter only the class-imbalance row and this split check; the default rates by group are from the training partition only

Table A4: Contribution by section. Percentages are of the writing and analysis behind each section; the model families below are owned end to end — data, tuning, interpretation and presentation — by the member named. The percentages were drafted on 21 October 2026 and confirmed by all three members on 27 October 2026; the overall row is the unweighted mean of the nine rows, rounded.

Section	RUAN	LU	LI	Basis
Abstract, §1 Introduction	30%	20%	50%	LI drafted; RUAN and LU revised
§2 Data	25%	15%	60%	LI wrote the cleaning and split contract and the exploratory analysis
§3 Methods	30%	25%	45%	shared protocol; RUAN the nested cross-validation, LI the freeze and recalibration protocol
§4 Results	30%	35%	35%	RUAN: KNN, logistic family, second pass; LU: tree, forest, GBM, XGBoost; LI: SVM, MLP, LDA/QDA/NB, imbalance study, aggregation and bootstrap
§5 Interpretation	15%	60%	25%	LU owns TreeSHAP, the monotone constraints, partial dependence and the forest importances
§6 Choosing the operating point	40%	15%	45%	RUAN: cost-ratio table and nested CV; LI: recalibration and threshold analysis
§7 Conclusion and limitations	34%	33%	33%	drafted jointly
Code and reproducibility	20%	25%	55%	LI: `common.py`, aggregation, freeze, recalibration
Presentation	33%	33%	34%	three presenters, 10 minutes
Overall (mean of rows)	29%	29%	42%	agreed by all three members

Each model family is run end to end by its owner: **RUAN Hangjing** — KNN benchmark; logistic family (plain, ridge, lasso, class-weighted); PCA and K-means; **LU Xinyu** — Pruned tree; random forest and bagging; gradient boosting and XGBoost; TreeSHAP; monotone-constrained XGBoost; partial dependence; **LI Jingwen** — Support-vector machines; feed-forward neural network; LDA, QDA and naive Bayes; class-imbalance study and the leakage demonstration. The owner of every file in `results/` is recorded in its `member` field, so this table is checkable against the submitted code. This project is not shared with any other module.

Table A5: Model families, the lecture that introduces each one, and what each is in the study to test. The last three rows are beyond the syllabus.

Family (lecture)	What it is there to test
PCA, K-means, hierarchical clustering (B2–B3)	Whether default risk lies along the directions of largest variance, and whether a segment label carries anything a supervised model has not already found.
K-nearest neighbours (B4)	Benchmark: how much of the signal a scaled distance and a local default rate recover with no model of the risk at all.
Class weights, SMOTE, under-sampling (B4)	Whether re-balancing changes the ordering of clients or only the prior behind it; and what resampling before the split fabricates.
Logistic regression: plain, ridge, lasso (B5–B6)	Whether the ceiling on this data is variance, which shrinkage can cut, or how the inputs are coded and shaped (the status codes read as one evenly spaced number), which it cannot.
LDA, QDA, naive Bayes (B6)	The price of distributional assumptions: a shared covariance, class-specific covariances on skewed amounts, and independence among near-collinear balances.
Decision tree, cost-complexity pruning (B7)	The bias–variance trade-off made readable: delay thresholds and their interactions, bought at the cost of very few distinct scores.
Random forest and bagging (B7)	Whether de-correlating trees, not merely averaging them, is what removes variance; out-of-bag error and permutation importance come free.
Gradient boosting and XGBoost (B7)	Whether sequential bias reduction beats variance reduction on a noisy target, and how much of that depends on shrinkage.
Support-vector machines, linear and RBF (B8)	Whether a margin loss that ignores ordering beyond the margin can rank risk as well as a proper scoring rule.
Feed-forward neural network (B9)	Whether learned interactions help when the strongest predictors are repayment-status codes with step-like effects.
TreeSHAP (beyond the syllabus)	Direction and size of each variable's push on each individual client, rather than one global ranking; additive by construction, so it can be checked.
Monotonically constrained XGBoost (beyond the syllabus)	The cost, in ranking quality, of the scorecard requirement that more arrears may never lower predicted risk.
Partial dependence and ICE (beyond the syllabus)	Whether the fitted surface carries the U-shaped utilisation effect a linear term cannot express, and whether effects add or interact.

Table A6: Hyper-parameter grids searched for the headline models and the setting chosen, read from `results/<slug>.json`. Every grid is scored by 5-fold stratified cross-validation on the training rows shown (seed 2607526); the test set plays no part. Rule: argmax of CV AUC, or the one-standard-error rule (1-SE). The last column is the training-set selection score of the chosen setting. The reported CV is the maximum over the grid on the same folds, an upper estimate for close winners. Gradient boosting was tuned on a training subsample, not fixed a priori; only the monotone constraint vector and bagging's inherited leaf size were fixed a priori.

Model	Grid searched	Chosen	Rows	Settings	Rule	Sel. AUC
XGBoost	`max_depth` {3, 4, 6} × `learning_rate` {0.03, 0.1} × `n_estimators` {300, 600} × `min_child_weight` {5, 1} (24 settings); `subsample` = 0.8, `colsample_bytree` = 0.8 fixed; rounds then checked on a learning curve to 1500	`max_depth` = 3, `learning_rate` = 0.03, `n_estimators` = 300, `min_child_weight` = 5	20,975	24	argmax	0.7863
Gradient boosting (sklearn)	`learning_rate` {0.03, 0.1} × `max_depth` {3, 2, 4} × `n_estimators` {200, 500} (12 settings) on a 10,487-row training subsample; staged CV curve on the full training set	`learning_rate` = 0.03, `max_depth` = 3, `n_estimators` = 200, `subsample` = 0.8	10,487	12	argmax	0.7850
Random forest	`max_features` {sqrt, 0.3, 0.5} × `min_samples_leaf` {20, 5, 1} (9 settings) at 200 trees; top two re-run at 500 trees	`max_features` = sqrt, `min_samples_leaf` = 20, `n_estimators` = 500	20,975	9	argmax	0.7849
MLP (feed-forward NN)	`hidden_layer_sizes` {32, 64, 64x32, 128x64} × `alpha` {0.0001, 0.001, 0.01} × `learning_rate_init` {0.001, 0.003} (24 settings); `relu`, `adam`, early stopping fixed	`hidden_layer_sizes` = 64x32, `alpha` = 0.001, `learning_rate_init` = 0.001	20,975	24	argmax	0.7762
KNN	`K` {5 … 151} (12 values) × `weights` {uniform, distance} (24 settings)	`n_neighbors` = 101, `weights` = uniform	20,975	24	argmax	0.7700
Decision tree (pruned)	`ccp_alpha` {0 … 0.02493} (21 values) (cost-complexity path) × `max_depth` {None, 4, 6, 8}	`criterion` = entropy, `ccp_alpha` = 0.000663, `max_depth` = 6	20,975	84	argmax	0.7689
SVM (RBF kernel)	`C` {0.1, 1, 10} × `gamma` {scale, 0.01, 0.1} × `class_weight` {None, balanced} on a 6,000-row training subsample; one-standard-error rule	`C` = 1, `gamma` = 0.01, `class_weight` = balanced; Platt-scaled	6,000	18	1-SE	0.7635[†]
Logistic regression (plain)	none: unpenalised reference fit	`C` = inf (unpenalised), `solver` = lbfgs	20,975	1	none	0.7602
Logistic regression (L1 lasso, C tuned)	`C` {0.001 … 100} (11 values)	`C` = 100, `penalty` = l1 (l1_ratio=1)	20,975	11	argmax	0.7602
LDA (linear discriminant analysis)	`shrinkage` {None, auto, 0.05, 0.2, 0.5}	`solver` = lsqr, `shrinkage` = None	20,975	5	argmax	0.7572
Gaussian naive Bayes	`var_smoothing` {1e-09 … 0.1} (9 values)	`var_smoothing` = 0.01	20,975	9	argmax	0.7356

[†]Tuned on a 6,000-row training subsample; the score shown is the mean of the five shared-fold out-of-fold AUCs of the chosen setting refitted on the full training partition (pooled out-of-fold 0.761, subsample CV 0.768; `results/svm_selection_statistics.json`). Post-freeze checks of these grids: Table A24.

Table A7: Software versions and the single seed behind every result in this report. Versions are the ones recorded in the result files by the scripts that produced them.

Component	Version / value
Python	3.13.5
scikit-learn	1.9.0
XGBoost	3.4.1
SHAP	0.52.0
NumPy	2.5.1
pandas	3.0.5
imbalanced-learn	0.14.2
SciPy	1.18.0
Random seed (split, folds, samplers, every model)	2607526
Cross-validation	5 stratified folds of the training partition, shuffled with the seed
Split	stratified 70/30, 20,975 training / 8,990 test rows

Table A8: The monotone constraint vector of the constrained XGBoost, fixed a priori from credit logic and read back from the fitted booster: `(-1,0,1,1,1,1,1,1,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,1,-1,1,1,1,0,0,0,0,0,0)` over the 32 preprocessed columns (one-hot categories count once per raw variable here). Hyper-parameters are inherited unchanged from the tuned unconstrained model.

Sign	Variables	n
$+1$ (more may only raise the predicted risk)	PAY_1, PAY_2, PAY_3, PAY_4, PAY_5, PAY_6, N_MONTHS_DELAYED, MAX_DELAY, UTIL_1, UTIL_MEAN, BILL_TREND	11
-1 (more may only lower it)	LIMIT_BAL, PAY_RATIO_MEAN	2
0 (unconstrained)	AGE, BILL_AMT1, BILL_AMT2, BILL_AMT3, BILL_AMT4, BILL_AMT5, BILL_AMT6, PAY_AMT1, PAY_AMT2, PAY_AMT3, PAY_AMT4, PAY_AMT5, PAY_AMT6, SEX, EDUCATION, MARRIAGE	19

The constraint imposes an order on the non-delinquent codes $-2/-1/0$ and a rising utilisation that the training data contradict (Table A29, which also gives the wrong-direction steps of the unconstrained booster under a consistent sweep). Its price is -0.0019 CV AUC (0/5, $[-0.0025, -0.0013]$); test AUC -0.0016 as a check (paired bootstrap, $B = 500$: mean -0.0017, 95 % interval $[-0.0030, -0.0002]$).

Table A9: Freeze timeline: the 46 entries of `results/frozen_manifest.json` grouped by round. A configuration is frozen the first time it is scored on the test set, and `common.evaluate()` refuses to score a changed configuration under a frozen slug (`code/freeze.py --check` verifies every stored prediction vector). The manifest lists configurations only; every read of the test set, including descriptive reads outside it, is dated in `results/TEST_SET_LOG.md`.

Round	Scored	n	What	Selection basis
1	16 Sep–18 Sep	34	every model variant of Table A12: each configuration tuned on training CV, then scored on the test set (before the professor's 30 September comment) and registered on 19 October; the forest and bagging were scored twice with the same configuration (16 and 18 September); dates are result-file times	5-fold CV AUC on the training partition; both thresholds on training out-of-fold probabilities
2	30 Sep	2	RUAN Hangjing's second pass under her freeze-before-test protocol (`frozen_choices_ruan_v2.json`): the KNN ($K = 101$, uniform) and plain-logistic configurations re-fitted and re-scored; adopted on 21 October unchanged; the headline tables keep the round-1 scoring (predictions differ by $\leq 10^{-2}$, test AUC by 5×10^{-5})	nested CV with the pre-registered 1-SE and 1 % cost rules (Table A11); training rows only
3	19 Oct	9	Platt-recalibrated versions of the nine class-weighted or resampled models (Table A16), each registered before its 19 October scoring (a 6 October review had already applied the same maps to the test scores)	Platt map fitted on training out-of-fold scores; thresholds re-chosen on cross-fitted training scores
—	17 Sep	1	negative control: SMOTE applied before the split (the leakage demonstration); it pools the training and test rows, so its re-split 'test' rows are not the real test set, and it is not a model result	none — deliberately leaky

Table A10: Every read of the test rows, condensed from `results/TEST_SET_LOG.md` (section ids in the first column). Each configuration was frozen at its first scoring; the manifest of Table A9 lists configurations, this log lists reads. The only choice made after a test read is the linear SVM's loss swap, whose stated basis is training CV; until 19 October test AUC set two presentation choices of the aggregation.

Section	When	What	Influenced a choice?
A1	16–18 Sep	Round 1: 34 configurations and 1 negative control (SMOTE before the split, which pools the real test rows), scored before any freeze existed and registered on 19 October; the forest and bagging were scored twice (16 and 18 September) with the same configuration	Modelling choices: no (hyper-parameters and cut-offs from training CV / out-of-fold scores). Until 19 October test AUC set two presentation choices of the aggregation: the bootstrap's reference model and the four models drawn in three figures
A2	30 Sep	RUAN Hangjing's second pass: 2 re-fits of frozen configurations (`knn_v2`, `logistic_plain_v2`), with reporting reads (cost sensitivity, saving bootstrap)	No: selections by nested CV on training rows
A3	19 Oct	9 Platt-recalibrated variants, each registered before its scoring; the same maps had been applied to the test scores in the 6 October review (B8)	No: Platt was pre-committed in the 30 September proposal
B1–B12	16 Sep–6 Oct	Reads outside the manifest: a plain-logit reference with $C = 1$ (B1); the hinge-loss linear SVM (B2); test AUC against the number of trees (B4); the monotone model's twin and bootstrap (B5); six drop-one-column forests and a third forest scoring (B6); RUAN's proposal-stage logit (B7); the 6 October review's recalibration maps and bootstraps (B8); descriptive reads in four scripts (B9–B12)	Once: the linear SVM's hinge-to-squared-hinge loss swap (B2) came after the hinge model's test score; its stated basis is training CV. Otherwise no
C	16 Sep–21 Oct	Reporting reads of frozen predictions: aggregation (curves, bootstrap), threshold against model saving, cost sensitivity (re-run on the 0.01 cut-off grid on 21 October), the split check, slide figures and the 21 October fresh-clone reproduction	No: reporting only
D	16 Sep–21 Oct	Test features without labels: reproduction checks of frozen predictions, feature counts in Table A3, calls that open the test file and discard it	No
F	21 Oct	Activity that did not read the test set: the post-freeze checks of Section A.2 and the R cross-check (Table A33)	—

Table A11: Nested cross-validation of the KNN and logistic families, RUAN Hangjing's second pass of 30 September 2026 (`logistic_nested_cv_v2.csv`, `knn_v2.json`), plus the KNN re-tuned inside each outer fold over the same 24-setting grid as the frozen model (`nested_cv_knn_frozen_grid.json`, 21 October). The earlier 16-setting KNN run is RUAN's 30 September method, reproduced by `nested_cv_check.py`. The 21 October reproduction on the training rows (`nested_cv_summary_repro.json`, `logistic_nested_cv_repro.csv`) matched the nested AUCs and standard errors, the logistic rows and the selected configurations; the KNN Brier and calibration-slope columns are RUAN's 30 September out-of-fold figures (`knn_v2.json`), which the reproduction (`nested_cv_knn_repro.json`) puts at 0.1395 and 0.966. The pre-registered rules re-select the configurations frozen on 16 September. Only KNN and the logistic family were nested; the ensembles were tuned by plain 5-fold CV.

Model	Nested AUC	Nested AP	Brier	Cal. slope	5:1 cut-off	5:1 cost
Logistic, plain (unpenalised)	0.7602 ± 0.0050	0.5049	0.1410	0.982	0.17	12,195
Logistic, ridge (C tuned inside each fold)	0.7601 ± 0.0051	0.5049	0.1410	0.985	0.17	12,205
Logistic, lasso (C tuned inside each fold)	0.7602 ± 0.0050	0.5049	0.1410	0.982	0.15	12,188
Logistic, class-weighted	0.7608 ± 0.0055	0.5020	0.1920	0.974	0.39	12,263
Logistic, class-weighted + Platt	0.7608 ± 0.0055	0.5022	0.1412	1.004	0.15	12,247
KNN, RUAN's 16-setting grid (K, weights tuned in-fold; $K = 101$ uniform on the full training set)	0.7684 ± 0.0073	—	0.1392	0.989	—	—
KNN, the frozen 24-setting grid (outer-fold choices: $K = 101$ in 3; $K = 151$ in 2; distance weights in 2 of 5)	0.7684 ± 0.0073	—	—	—	—	—

Nested CV: outer loop = the project's five folds, inner loop = a 5-fold grid search inside each outer training fold, so the AUC is that of the whole tuning procedure. Columns: nested AUC $\pm$ its standard error over the outer folds (for KNN, the standard deviation over the outer folds, ddof 0; with ddof 1, as in Table 1, the 24-setting run gives 0.0082), nested average precision, Brier score and calibration slope of the nested out-of-fold scores, the 5:1 cut-off chosen on those scores (closed form $1/6 = 0.167$) and the cross-fitted 5:1 cost on the 20,975 training rows. Pre-registered rule: 1-SE on nested AUC among plain > ridge > lasso; adopt Platt-calibrated class weighting only if its cross-fitted 5:1 cost is > 1% lower. Outcome: plain kept; the Platt-calibrated weighting is 0.43 % costlier (12,247 against 12,195), so the rule (adopt only if $> 1\,\%$ cheaper) is not met and it is not adopted. For comparison, the plain 5-fold out-of-fold 5:1 cost of the frozen 16 September logit (`results/logistic_p_oof.npy` against the 20,975 training labels) is 12,054 at 0.15 and 12,064 at 0.17 (+0.08 %): a flat stretch of the cost curve, and a different cross-fit from the nested one, which is why the two costs at 0.17 differ. RUAN's 30 September refit of the same plain logit (`logistic_plain_v2.json`) gave a test cost of 7,065 at 0.5 and a training out-of-fold cut-off of 0.17 costing 5,141, against 7,063 / 0.15 / 5,126 for the 16 September scoring reported throughout; the proposal's saving table is that refit (note to Table A15). The nested-CV columns use training rows only; the test costs in this note are those of the frozen 16 and 30 September scorings (Table A10, A1–A2).

Table A12: Every model variant fitted in this project (34 in total), ordered by CV AUC, the training-set selection score: the mean of the five 5-fold out-of-fold AUCs on the training partition. The remaining columns are the frozen configurations' test-set scores, reporting only and not used to rank (every read is dated in Table A10). The imbalance variants use fixed, untuned learners (logit $C = 1$, forest minimum leaf 5). Four round-1 variants (`knn_k5`, `mlp_overfit`, `naive_bayes_log`, `tree_full`) sit at the 0.05 floor of `common.tune_threshold`. RUAN Hangjing's 30 September re-scoring of the KNN and plain-logistic configurations (Table A9) is not a separate model and is not listed.

Model	CV AUC	Test AUC	Test AP	Test Brier
XGBoost	0.786	0.789	0.578	0.131
XGBoost (class-weighted)	0.786	0.789	0.579	0.177
Gradient boosting (sklearn)	0.785	0.787	0.576	0.131
Random forest	0.785	0.787	0.576	0.132
XGBoost (monotone constraints)	0.784	0.787	0.573	0.132
Bagging (RF, max_features=None)	0.781	0.783	0.571	0.133
Random forest (500 trees) - class_weight='balanced'	0.781	0.782	0.569	0.161
Random forest (500 trees) - none	0.780	0.782	0.569	0.133
Random forest (500 trees) - under-sampling in folds	0.779	0.781	0.558	0.185
MLP (feed-forward NN)	0.776	0.773	0.551	0.135
Random forest (500 trees) - SMOTE in folds	0.774	0.774	0.554	0.153
MLP (tuned, balanced sample weights)	0.774	0.771	0.535	0.185
KNN	0.770	0.771	0.542	0.136
Decision tree (pruned)	0.769	0.776	0.535	0.133
SVM (RBF kernel)†	0.764	0.762	0.494	0.145
LinearSVC + Platt calibration	0.761	0.763	0.515	0.139
Logistic regression (plain, class_weight=balanced)	0.761	0.763	0.517	0.190
Logistic (C=1) - class_weight='balanced'	0.761	0.763	0.517	0.190
Logistic regression (L2 ridge, C tuned)	0.760	0.764	0.523	0.138
Logistic regression (plain)	0.760	0.764	0.523	0.138
Logistic regression (L1 lasso, C tuned)	0.760	0.764	0.523	0.138
Logistic (C=1) - none	0.760	0.764	0.523	0.138
Logistic (C=1) - SMOTE in folds	0.760	0.762	0.516	0.191
Logistic + K-means segment dummies (round-1 scoring, $K = 4$ first pass; not adopted by the pre-registered rule; the $K = 5$ decision came from repeated CV and was never test-scored)	0.760	0.763	0.522	0.138
Logistic (C=1) - under-sampling in folds	0.760	0.763	0.515	0.190
LDA (linear discriminant analysis)	0.757	0.759	0.525	0.139
Logistic regression on principal components only	0.755	0.762	0.518	0.139
Gaussian naive Bayes (log1p amounts)	0.754	0.756	0.508	0.193
QDA (quadratic discriminant analysis)	0.751	0.755	0.516	0.166
SVM (linear kernel)†	0.749	0.751	0.506	0.148
Gaussian naive Bayes	0.736	0.740	0.504	0.203
KNN (K=5)	0.706	0.709	0.421	0.161
MLP over-parameterised (256,256)	0.692	0.678	0.400	0.212
Decision tree (unpruned full tree)	0.615	0.613	0.290	0.272

†Tuned on a 6,000-row training subsample (changed on 21 October): the CV AUC shown is the mean of the five shared-fold out-of-fold AUCs of the chosen setting refitted on the full training partition, the same statistic as every other row (`results/svm_selection_statistics.json`); the earlier table showed the pooled out-of-fold AUC. SVM (RBF kernel): pooled out-of-fold 0.761, subsample CV 0.768; SVM (linear kernel): pooled out-of-fold 0.717, subsample CV 0.763.

Table A13: Operating points of the 11 headline configurations of `aggregate_summary.json` (the lasso logit duplicates the plain logit to four decimals). Both thresholds are chosen on training out-of-fold probabilities (maximum F_1; minimum 5:1 cost); recall, precision and the share of test clients flagged are at the cost threshold (`at_tuned_cost` of each frozen `<slug>.json`); the cost columns are the expected test cost at a 5:1 false-negative to false-positive ratio. Frozen configurations only; reporting, nothing is selected here.

Model	F_1 thr.	Cost thr.	Recall	Precision	Flagged (%)	Cost @0.5	Cost @F_1	Cost @ cost thr.
KNN	0.25	0.16	0.75	0.35	46.7	6,854	5,224	5,219
Logistic regression (plain)	0.30	0.15	0.74	0.37	44.4	7,063	5,417	5,126
Logistic regression (L1 lasso, C tuned)	0.30	0.15	0.74	0.37	44.4	7,064	5,417	5,125
LDA (linear discriminant analysis)	0.27	0.12	0.70	0.39	40.0	6,618	5,352	5,181
Gaussian naive Bayes	0.41	0.25	0.64	0.42	33.9	5,422	5,385	5,337
Decision tree (pruned)	0.28	0.16	0.76	0.36	46.5	6,933	5,308	5,085
Random forest	0.31	0.17	0.76	0.37	45.4	6,598	5,293	4,998
Gradient boosting (sklearn)	0.26	0.14	0.80	0.34	52.2	6,630	5,186	5,060
XGBoost	0.27	0.15	0.78	0.36	48.8	6,642	5,164	4,981
SVM (RBF kernel)	0.42	0.12	0.69	0.40	38.2	9,316	5,355	5,155
MLP (feed-forward NN)	0.27	0.19	0.71	0.39	40.7	6,625	5,282	5,136

Trivial rules for reference: flagging every client costs 7,001, flagging none costs 9,945.

Table A14: Cost-ratio sensitivity: total test cost of each model at its own cut-off, chosen on training out-of-fold probabilities, as the false-negative to false-positive ratio varies (reporting only; nothing is selected on these numbers).

Model	1:1	2:1	3:1	5:1	8:1	10:1	20:1
Gradient boosting	1,566	2,617	3,562	5,060	5,910	6,278	6,856
KNN	1,638	2,720	3,662	5,219	6,221	6,525	6,898
Logistic regression	1,659	2,772	3,677	5,126	6,381	6,697	6,993
MLP	1,594	2,711	3,664	5,136	6,121	6,468	6,998
Pruned tree	1,573	2,685	3,582	5,085	6,027	6,359	7,001
Random forest	1,573	2,671	3,559	4,998	5,992	6,257	6,850
SVM (RBF)	1,684	2,726	3,637	5,155	6,364	6,702	7,088
XGBoost	1,572	2,649	3,550	4,981	5,892	6,277	6,843
Cheapest on training OOF	GBM	RF	XGB	XGB	—	RF	KNN

Pruned tree at 20:1: the cut-off flags every client, so its cost is the flag-everyone rule (7,001). Cost anchors for the ratio (`results/cost_anchors.json`): a missed defaulter loses 50–70% of the median defaulter's bill (NT$19,868); a false alarm forgoes one to three months of a 1.5% monthly margin on the median good client's bill (NT$22,914.5), so the ratio runs from $0.5 \times 19{,}868/(3 \times 0.015 \times 22{,}914.5) = 9.6$ to $0.7 \times 19{,}868/(0.015 \times 22{,}914.5) = 40.5$: an upper bound, as the label is one missed payment and some defaulters later pay. Last row: the cheapest of the eleven headline models on pooled training out-of-fold scores at the same ratio (`results/preferred_model_by_ratio_oof.csv`; no test read; Table A31; — where that file has no such ratio); XGB = XGBoost, GBM = gradient boosting, RF = random forest.

Table A15: Threshold saving against model saving at nine false-negative to false-positive cost ratios r. Threshold saving: expected test cost of the frozen plain logistic at the 0.5 cut-off minus its cost at the cut-off chosen on training out-of-fold scores. Model saving: the logistic at its own cut-off minus the frozen XGBoost at its own cut-off. Brackets are 95 % paired-bootstrap intervals. Flagged: share of the test clients the logistic flags at its cut-off. Source: `results/threshold_vs_model_saving.csv` (Table 2 in the body is the excerpt). Frozen configurations only; reporting, nothing is selected here.

r	$1/(1+r)$	Logit cut-off	XGB cut-off	Logit cost @0.5	Flagged (logit, %)	Threshold saving	Model saving	XGB saving vs best blanket
1:1	0.500	0.50	0.48	1,659	10.5	0	87	417
						[0, 0]	[44, 131]	(21.0 %)
2:1	0.333	0.33	0.32	3,010	22.2	238	123	1,329
						[141, 338]	[65, 185]	(33.4 %)
3:1	0.250	0.30	0.27	4,361	24.3	684	127	2,417
						[541, 830]	[47, 214]	(40.5 %)
4:1	0.200	0.19	0.19	5,712	33.4	1,228	167	2,684
						[1,003, 1,450]	[46, 287]	(38.3 %)
5:1	0.167	0.15	0.15	7,063	44.4	1,937	145	2,020
						[1,604, 2,218]	[−18, 299]	(28.9 %)
7:1	0.125	0.12	0.13	9,765	65.1	3,682	442	1,360
						[3,186, 4,146]	[218, 649]	(19.4 %)
10:1	0.091	0.10	0.09	13,818	79.9	7,121	420	724
						[6,395, 7,791]	[187, 646]	(10.3 %)
15:1	0.062	0.06	0.06	20,573	95.7	13,507	450	385
						[12,415, 14,473]	[243, 652]	(5.5 %)
20:1[†]	0.048	0.01	0.05	27,328	99.9	20,335	150	158
						[18,909, 21,621]	[−52, 333]	(2.3 %)

Paired bootstrap of the 8,990 test rows, $B = 1000$, seed 2607526; the test set is read for these intervals but selects nothing (both cut-offs are chosen on training out-of-fold scores; the closed form is $1/(1+r)$). The model-saving interval includes zero at 5:1, 20:1. [†]At 20:1 the logistic cut-off flags 99.9 % of clients, i.e. the trivial flag-everyone rule, so the threshold 'saving' there is not a model result. Proposal-stage table (RUAN Hangjing's 30 September refit of the same plain logit, cut-off 0.17): threshold saving 1,924 [1,634, 2,193], model saving 160 [−8, 331] at 5:1. Last column: the cheaper blanket rule at each ratio (flag nobody up to 3:1, flag everybody above) minus the frozen XGBoost at its own cut-off, with the saving as a share of the blanket cost; the same frozen test predictions, nothing re-scored. Training-row version against no-parameter arrears rules: Table A30.

Table A16: Platt recalibration of the nine class-weighted or resampled models. The Platt map is fitted on each model's training out-of-fold scores; the 5:1 cut-off is re-chosen on cross-fitted training scores; each recalibrated version was then registered, scored on the test set (19 October) and frozen; a 6 October review had already applied the same maps to the test scores, but Platt was the method committed in the 30 September proposal. Columns: mean predicted probability on test (prevalence 0.221); test Brier score, with the unweighted twin in brackets; 5:1 cut-off; test cost at the closed-form threshold 1/6.

Model	Mean p	Brier (twin)	5:1 cut-off	Cost at $1/6$
Logistic, class-weighted	0.440 → 0.221	0.1901 → 0.1389 (0.1383)	0.39 → 0.15	6,828 → 5,093
Logistic (C=1), class-weighted	0.440 → 0.221	0.1901 → 0.1389 (0.1383)	0.41 → 0.15	6,829 → 5,086
Logistic (C=1), SMOTE in folds	0.440 → 0.222	0.1909 → 0.1391 (0.1383)	0.41 → 0.18	6,732 → 5,138
Logistic (C=1), under-sampled	0.441 → 0.221	0.1904 → 0.1390 (0.1383)	0.41 → 0.15	6,831 → 5,086
Forest, class-weighted	0.371 → 0.220	0.1606 → 0.1335 (0.1332)	0.36 → 0.18	5,860 → 5,023
Forest, SMOTE in folds	0.339 → 0.222	0.1529 → 0.1359 (0.1332)	0.32 → 0.18	5,679 → 5,124
Forest, under-sampled	0.432 → 0.222	0.1850 → 0.1345 (0.1332)	0.44 → 0.18	6,201 → 5,020
XGBoost, class-weighted	0.423 → 0.220	0.1772 → 0.1311 (0.1311)	0.37 → 0.14	6,251 → 4,950
MLP, balanced sample weights	0.433 → 0.227	0.1855 → 0.1377 (0.1353)	0.42 → 0.19	6,608 → 5,088

Table A17: Choice of K for K-means on the first 10 principal components of the 32 preprocessed columns (81.8 % of the variance; 2, 5 and 15 components carry 48, 65 and 92 %), training partition only. RUAN Hangjing's analysis of 30 September 2026 (`kmeans_selection_v2.csv`), reproduced on 21 October (`kmeans_selection_repro.csv`, identical). Bold: the K with the highest silhouette, chosen by the rule fixed in advance.

K	Silhouette	Bootstrap ARI (mean)	ARI 5th pct.	Min. cluster Jaccard
2	0.260	0.986	0.971	0.993
3	0.267	0.987	0.976	0.980
4	0.257	0.976	0.939	0.974
5	**0.269**	0.941	0.745	0.768
6	0.209	0.787	0.544	0.670
7	0.215	0.957	0.905	0.889
8	0.218	0.954	0.863	0.404

Two bootstrap runs, not one. The three right-hand columns come from the selection sweep over K (for $K = 5$: ARI 0.941, minimum cluster Jaccard 0.768). The stability figures in Table A18 come from the separate 100-refit stability run of the chosen $K = 5$ (`unsupervised_summary_v2.json`): ARI 0.957 [0.745, 0.990] under bootstrap resampling, per-cluster Jaccard 0.972/0.974/0.791/0.970/0.953. Across random starts (single-start refits with other seeds on the same rows, against the reference partition) the groups shift more: ARI mean 0.81, minimum 0.58; agreement with Ward's hierarchical clustering at the same K: ARI 0.45–0.59 over five bootstrap samples (moderate).

Table A18: Profiles of the 5 K-means segments on the training partition (RUAN Hangjing, 30 September 2026, `kmeans_profiles_v2.csv`; reproduced 21 October). Jaccard: per-cluster stability over the bootstrap refits. The default rates differ because the inputs contain the delay and limit variables; the segments are descriptive, not economically distinct.

Cluster	n	Share	Default rate	Mean limit (NT$)	Months delayed	Utilisation	Jaccard
0	7,977	38.0%	16.8%	214,213	0.30	0.04	0.972
1	2,619	12.5%	58.5%	79,798	4.32	0.58	0.974
2	886	4.2%	6.9%	324,323	0.15	0.22	0.791
3	7,560	36.0%	17.9%	98,695	0.35	0.57	0.970
4	1,933	9.2%	18.2%	285,867	0.53	0.75	0.953

Incremental value of the one-hot segment label as an extra predictor of the plain logit, 5×5 repeated CV on the training rows with the Nadeau–Bengio corrected interval: ΔAUC $+0.0001$ $[-0.0004, +0.0007]$, ΔAP -0.0003 $[-0.0016, +0.0010]$; adoption rule (both lower bounds > 0) not met, adopted = False, so the test set was not consulted for it. The label used as a score alone has AUC 0.648 against 0.724 for months-in-arrears alone; the logit on the 10 PCs alone changes AUC by -0.034 against the logit on the raw columns.

Table A19: Four ways to rank the same 29 predictors: mean |SHAP| on the tuned XGBoost over the 20,975 training clients, and the random forest's impurity decrease, training-set permutation and out-of-bag permutation importances. Ranks, not magnitudes, are comparable across columns.

Feature	mean |SHAP|	rank	MDI	rank	perm. (train)	rank	perm. (OOB)	rank
PAY_1	0.2980	1	0.1876	1	0.0423	1	0.0114	1
MAX_DELAY	0.2815	2	0.1018	2	0.0145	7	0.0009	7
N_MONTHS_DELAYED	0.2303	3	0.0935	3	0.0097	17	0.0000	19
BILL_AMT1	0.1128	4	0.0274	12	0.0158	5	0.0018	3
UTIL_MEAN	0.1100	5	0.0368	6	0.0187	2	0.0021	2
LIMIT_BAL	0.1024	6	0.0264	13	0.0163	4	0.0017	4
PAY_AMT1	0.0858	7	0.0299	9	0.0158	6	0.0007	8
UTIL_1	0.0709	8	0.0313	7	0.0185	3	0.0010	6
BILL_TREND	0.0709	9	0.0292	10	0.0134	9	0.0004	10
PAY_AMT3	0.0667	10	0.0233	17	0.0123	10	0.0002	15
PAY_AMT2	0.0646	11	0.0240	15	0.0139	8	0.0011	5
MARRIAGE	0.0600	12	0.0045	29	0.0034	26	0.0002	14
BILL_AMT5	0.0484	13	0.0222	18	0.0108	14	−0.0000	23
PAY_AMT4	0.0441	14	0.0212	19	0.0119	11	0.0007	9
SEX	0.0436	15	0.0048	28	0.0035	25	0.0004	11
EDUCATION	0.0319	16	0.0054	27	0.0041	24	0.0001	18
AGE	0.0220	17	0.0191	22	0.0078	21	0.0003	12
BILL_AMT3	0.0207	18	0.0205	20	0.0108	15	0.0002	16
PAY_AMT5	0.0190	19	0.0178	25	0.0093	19	−0.0007	28
PAY_AMT6	0.0189	20	0.0182	24	0.0102	16	0.0001	17
PAY_2	0.0168	21	0.0558	4	0.0064	22	0.0000	22
PAY_RATIO_MEAN	0.0165	22	0.0303	8	0.0118	12	−0.0001	25
BILL_AMT4	0.0160	23	0.0201	21	0.0095	18	−0.0003	27
PAY_3	0.0124	24	0.0376	5	0.0047	23	−0.0001	24
BILL_AMT2	0.0106	25	0.0236	16	0.0117	13	0.0002	13
BILL_AMT6	0.0089	26	0.0184	23	0.0081	20	−0.0008	29
PAY_5	0.0053	27	0.0243	14	0.0028	28	0.0000	20
PAY_4	0.0033	28	0.0277	11	0.0033	27	−0.0002	26
PAY_6	0.0025	29	0.0175	26	0.0027	29	0.0000	21

Spearman rank correlation over the 29 predictors (`random_forest.json` `extra.spearman`, `shap_summary.json` `spearman_vs_other_measures`): MDI vs out-of-bag permutation 0.310, MDI vs training-set permutation 0.538, training-set vs out-of-bag permutation 0.719; mean |SHAP| vs MDI 0.433, vs training-set permutation 0.756, vs out-of-bag permutation 0.772. The Spearman values between SHAP (XGBoost) and MDI or permutation (random forest) are cross-model: they compare two different fitted models, not two measures of one model.

Table A20: Refitting the tuned random forest without one column, for the six columns with the largest out-of-bag permutation drop. Permutation measures how much the fitted forest leans on a column; refitting measures what is lost if the column is never available. Out-of-bag AUC on the training rows; the test set is not used.

Column removed	OOB AUC	OOB AUC lost	OOB perm. drop
(none: full model)	0.7849	0.00000	—
PAY_1	0.7791	0.00577	0.0114
UTIL_MEAN	0.7849	−0.00004	0.0021
BILL_AMT1	0.7841	0.00076	0.0018
LIMIT_BAL	0.7846	0.00028	0.0017
PAY_AMT2	0.7839	0.00096	0.0011
UTIL_1	0.7849	−0.00002	0.0010

A.2 Post-freeze checks on the training rows (21 October 2026)

Every table in this subsection is a post-freeze sensitivity check: training five-fold CV on the shared folds (`data/processed/cv_folds.csv`, seed 2607526), run on 21 October 2026 after all configurations were frozen. Nothing here was scored on the test set, and nothing here selects a model, feature or cut-off. Paired comparisons give the mean of the five fold differences, a Nadeau–Bengio corrected 95 % interval ($n_{\text{test}}/n_{\text{train}} = 1/4$) in brackets and the number of folds in which the difference is positive (k/5). Source: `results/POST_REVIEW_CHECKS.md`. The figures of these checks are in Section A.3.

Table A21: What the repayment codes mean and how often they occur, training rows only ($n = 20{,}975$; `results/pay_code_default_rates.csv`). Each cell counts the training clients with that code in that month; the September default rate is the share of them who defaulted on the October 2005 payment. A status is compared within a month, not across months: the same code does not carry the same risk in different months.

		September (`PAY_1`)		Clients with the code in earlier months				
Code	Meaning	n	Default (%)	Aug	Jul	Jun	May	Apr
-2	no consumption[†]	1,904	13.7	2,590	2,777	2,995	3,138	3,378
-1	paid in full	3,941	16.9	4,221	4,188	4,013	3,883	4,046
0	revolving credit[†]	10,380	12.9	11,063	11,042	11,491	11,877	11,393
1	1 month late	2,540	33.9	17	3	1	—	—
2	2 months late	1,871	67.6	2,750	2,706	2,218	1,828	1,933
3	3 months late	234	77.4	227	156	132	125	134
4	4 months late	57	66.7	63	48	45	65	30
5	5 months late	16	56.2	14	19	31	11	12
6	6 months late	6	33.3	12	21	4	3	13
7	7 months late	9	77.8	18	14	44	45	35
8	8 months late	17	58.8	—	1	1	—	1

[†]Codes -2 and 0 are not defined in the UCI documentation; their readings are widely used, not documented. The delay counters (`MAX_DELAY`, `N_MONTHS_DELAYED`) treat -2, -1 and 0 all as 0. Code 1 is recorded almost only in September: of its 2,540 September clients, 1,173 (46.2 %) had no positive September bill, 1,156 (45.5 %) were two months late in August, 353 had six zero bills (40.2 % default) and 422 followed five months of -2. Share of September k-months-late clients who were two months late in August (a backward share: where September's clients came from, not how many August clients moved on): code 2 59.1 % (arrears that persisted), code 3 85.9 % (arrears that deepened one step in a month).

Table A22: Reading the repayment codes by meaning (post-freeze, training CV only, never test-scored). Five model families refitted with their frozen settings on three codings of `PAY_1`–`PAY_6`: the frozen single number per month; months late, $\max(\text{code}, 0)$, plus indicators for -2 and -1; and one dummy per code and month. Δ = this coding minus the numeric coding of the same model, fold-paired. SD over the five fold AUCs, ddof 1. Sources: `results/pay_coding_cv.csv`, `pay_coding_summary.json`, `pay_coding_logit_coefficients.csv`.

Model and coding	Inputs	CV AUC ± SD	ΔAUC, *k*/5 [NB]	CV AP	ΔAP, *k*/5 [NB]	Brier
Logit (plain)						
numeric (as frozen)	32	0.7602 ± 0.0112	—	0.5067	—	0.1410
months late + flags	44	0.7684 ± 0.0086	+0.0082, 5/5 [+0.0021, +0.0143]	0.5275	+0.0208, 5/5 [+0.0108, +0.0307]	0.1380
one-hot†	81	0.7691 ± 0.0087	+0.0089, 5/5 [+0.0031, +0.0146]	0.5318	+0.0251, 5/5 [+0.0094, +0.0408]	0.1374
Logit, L2 ($C = 31.6$)						
numeric (as frozen)	32	0.7602 ± 0.0112	—	0.5067	—	0.1410
months late + flags	44	0.7684 ± 0.0086	+0.0082, 5/5 [+0.0021, +0.0143]	0.5275	+0.0208, 5/5 [+0.0108, +0.0307]	0.1380
one-hot	81	0.7695 ± 0.0088	+0.0092, 5/5 [+0.0041, +0.0143]	0.5320	+0.0253, 5/5 [+0.0095, +0.0411]	0.1373
LDA						
numeric (as frozen)	32	0.7572 ± 0.0122	—	0.5065	—	0.1431
months late + flags	44	0.7651 ± 0.0095	+0.0079, 5/5 [+0.0007, +0.0150]	0.5265	+0.0200, 5/5 [+0.0113, +0.0288]	0.1404
one-hot	81	0.7676 ± 0.0092	+0.0103, 5/5 [+0.0032, +0.0175]	0.5323	+0.0258, 5/5 [+0.0113, +0.0404]	0.1412
KNN ($K = 101$)						
numeric (as frozen)	32	0.7700 ± 0.0097	—	0.5187	—	0.1392
months late + flags	44	0.7680 ± 0.0076	−0.0020, 2/5 [−0.0065, +0.0025]	0.5204	+0.0017, 4/5 [−0.0022, +0.0056]	0.1391
one-hot	81	0.7705 ± 0.0112	+0.0004, 3/5 [−0.0035, +0.0044]	0.5189	+0.0002, 3/5 [−0.0066, +0.0071]	0.1392
MLP (64–32)						
numeric (as frozen)	32	0.7762 ± 0.0077	—	0.5352	—	0.1373
months late + flags	44	0.7733 ± 0.0089	−0.0029, 0/5 [−0.0079, +0.0021]	0.5341	−0.0010, 2/5 [−0.0095, +0.0075]	0.1373
one-hot	81	0.7753 ± 0.0107	−0.0010, 2/5 [−0.0071, +0.0051]	0.5352	−0.0000, 3/5 [−0.0180, +0.0180]	0.1373

Numeric coding reproduces every frozen out-of-fold vector exactly. †The unpenalised one-hot logit stopped at `max_iter` in every fold (rare codes, some held by a single training client, make the maximum-likelihood estimate infinite); the L2 one-hot row is the converged fit of the same coding. Cross-model, fold-paired: recoded logit − KNN −0.0016 [−0.0080, +0.0048], 2/5; XGBoost − logit (numeric) +0.0261 [+0.0145, +0.0376], 5/5; XGBoost − recoded logit +0.0179 [+0.0106, +0.0251], 5/5; XGBoost − one-hot logit +0.0172 [+0.0095, +0.0249], 5/5. The recoding closes 84 % of the KNN–logit gap but only 31 % of the XGBoost–logit gap. Recoded plain logit, per unit, against revolving credit in September: paid in full +0.84, no consumption +0.61 log-odds, each month late +0.59; one more month late in August … April −0.42 … −0.11 log-odds per month, conditional on the delay counters (all five folds agree in sign). These coefficients describe associations in this vintage, not causes.

Table A23: Fold-by-fold stability of the eleven headline configurations: training out-of-fold scores of the frozen configurations on the five shared folds (`results/fold_stability.csv`); nothing re-fitted, nothing test-scored. Top: AUC in each validation fold (4,195 clients), mean ± SD (ddof 1) and the rank in each fold. Bottom: pooled out-of-fold average precision (AP) with its lift over the base rate (0.221), Brier score and calibration slope, and the fold-paired difference XGBoost − model with its Nadeau–Bengio interval and the Holm-adjusted p-value over the ten comparisons.

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean ± SD	Ranks
XGBoost	0.7930	0.7786	0.7796	0.7953	0.7850	0.7863 ± 0.0076	1 1 1 1 2
Gradient boosting	0.7920	0.7756	0.7786	0.7946	0.7841	0.7850 ± 0.0082	3 2 2 2 3
Random forest	0.7929	0.7740	0.7785	0.7929	0.7862	0.7849 ± 0.0085	2 3 3 3 1
MLP	0.7850	0.7668	0.7724	0.7834	0.7734	0.7762 ± 0.0077	4 4 4 4 4
KNN ($K = 101$)	0.7832	0.7576	0.7694	0.7750	0.7650	0.7700 ± 0.0097	5 6 5 7 6
Pruned tree (17 leaves)	0.7805	0.7604	0.7637	0.7750	0.7651	0.7689 ± 0.0084	6 5 6 5 5
SVM (RBF)†	0.7770	0.7463	0.7581	0.7750	0.7613	0.7635 ± 0.0127	7 7 7 6 7
Logistic (plain)	0.7752	0.7449	0.7573	0.7656	0.7581	0.7602 ± 0.0112	8 8 8 9 9
Logistic L1	0.7752	0.7449	0.7573	0.7656	0.7581	0.7602 ± 0.0112	9 9 9 8 8
LDA	0.7718	0.7398	0.7543	0.7652	0.7549	0.7572 ± 0.0122	10 10 10 10 10
Gaussian NB	0.7549	0.7282	0.7267	0.7408	0.7276	0.7356 ± 0.0122	11 11 11 11 11

Model	Pooled OOF AP (lift)	OOF Brier	Cal. slope	XGBoost − model mean, k/5	NB 95 % interval	Holm p
XGBoost	0.552 (2.50×)	0.1344	1.01	reference	—	—
Gradient boosting	0.551 (2.49×)	0.1346	1.04	+0.0013, 5/5	[−0.0005, +0.0031]	0.221
Random forest	0.551 (2.49×)	0.1348	1.03	+0.0014, 4/5	[−0.0027, +0.0055]	0.401
MLP	0.532 (2.40×)	0.1373	0.94	+0.0101, 5/5	[+0.0058, +0.0144]	0.022
KNN ($K = 101$)	0.517 (2.34×)	0.1392	0.99	+0.0163, 5/5	[+0.0055, +0.0270]	0.041
Pruned tree (17 leaves)	0.519 (2.34×)	0.1373	0.93	+0.0174, 5/5	[+0.0114, +0.0233]	0.011
SVM (RBF)†	0.485 (2.19×)	0.1464	1.00	+0.0228, 5/5	[+0.0115, +0.0340]	0.022
Logistic (plain)	0.505 (2.28×)	0.1410	0.98	+0.0261, 5/5	[+0.0145, +0.0376]	0.022
Logistic L1	0.505 (2.28×)	0.1410	0.98	+0.0261, 5/5	[+0.0145, +0.0376]	0.022
LDA	0.505 (2.28×)	0.1431	0.75	+0.0291, 5/5	[+0.0169, +0.0413]	0.022
Gaussian NB	0.487 (2.20×)	0.2088	0.14	+0.0507, 5/5	[+0.0368, +0.0645]	0.005

†SVM: mean of the five shared-fold out-of-fold AUCs (pooled 0.761; `svm_selection_statistics.json`). The frozen JSON `cv_auc_std` uses ddof 0; this table uses ddof 1. Blocks that hold in every fold: {XGBoost, GBM, forest} 1–3, MLP 4, {KNN, tree, SVM} 5–7, {logits} 8–9, LDA 10, naive Bayes 11. Other neighbours, fold-paired: MLP − KNN +0.0062 [−0.0003, +0.0127], 5/5; pruned tree − KNN −0.0011 [−0.0072, +0.0050], 3/5; SVM − plain logit +0.0033 [−0.0032, +0.0098], 5/5 (stored in `svm_selection_statistics.json`).

Table A24: Tuning-budget and grid checks (post-freeze, training CV only, never test-scored). Nothing was selected: every frozen configuration stays. The reported CV is the maximum over each grid on the same folds, so winners' gaps are upper estimates. Differences are against the frozen configuration of the same model. Sources: `results/gbm_grid_fulltrain.csv`, `tuning_checks_summary.json`, `nested_cv_knn_frozen_grid.json`, `knn_dimensionality_cv.csv`, `grid_edge_check.csv`, `svm_selection_statistics.json`.

Check	Setting	CV AUC	Difference k/5 [NB]	Reading
GBM grid re-scored on the full training set	12 settings on 20,975 rows (frozen grid was scored on a subsample); winner depth 4, learning rate 0.03, 200 trees; the frozen setting ranks 3 (0.7850)	0.7858	$+0.0008$, 5/5 $[-0.0003, +0.0019]$	Spearman with the subsample order 0.88; XGBoost − full-set winner $+0.0005$, 3/5; rule fixed beforehand: report only, keep the frozen GBM
KNN nested CV over the frozen 24-setting grid	outer folds = the five shared folds; choices: $K = 101$ in 3; $K = 151$ in 2; distance weights in 2	0.7684 ± 0.0073	-0.0016, 0/5 $[-0.0050, +0.0018]$	nested − plain CV of the frozen $K = 101$ (2 folds tie)
KNN in fewer dimensions	32 columns: 26 standardised, 6 one-hot (frozen)	0.7700	—	
	15 principal components (92 % of the variance)	0.7678	-0.0022, 1/5 $[-0.0083, +0.0039]$	PCA fitted inside each fold;
	10 principal components (82 % of the variance)	0.7644	-0.0056, 0/5 $[-0.0140, +0.0027]$	$K = 101$ uniform, not re-tuned
	5 principal components (65 % of the variance)	0.7603	-0.0098, 0/5 $[-0.0141, -0.0055]$	
	top 8 mutual-information columns (about 888 distinct points)	0.7295	-0.0405, 0/5 $[-0.0566, -0.0244]$	selector fitted inside each fold; all eight columns are repayment-status codes or counters, so many clients coincide
XGBoost grid extension	60 settings: depth 2–3 × learning rate 0.01–0.03 × 150–600 rounds; best depth 3, rate 0.03, 350 rounds	0.78637	$+0.00008$, 3/5 $[-0.0006, +0.0008]$	no setting beats the frozen one by one fold SD (0.0068); best at depth 2: 0.7853; rate 0.01 at 1,100 rounds 0.78657 ($+0.0003$)
Grid edges not probed	forest `max_features` sqrt; forest `min_samples_leaf` 20 (CV still rising: 0.7666 / 0.7796 / 0.7843); GBM `learning_rate` 0.03; XGBoost `min_child_weight` 5; MLP `learning_rate_init` 0.001; QDA `reg_param` 0.5 (upper edge; CV 0.7394 / 0.7429 / 0.7495 / 0.7511)	—	—	the chosen value sits on the edge of its grid and nothing beyond it was tried
SVM selection statistic	RBF: subsample CV 0.768, pooled OOF 0.761, mean of fold AUCs 0.764; linear: 0.763, 0.717, 0.749	0.7635	—	rank 7 of 11 under each statistic

Table A25: Interaction depth (post-freeze, training CV only, never test-scored): CV AUC of XGBoost with the frozen settings and only the tree depth changed (in the re-tuned columns also the number of rounds), against the plain logit, on the frozen 29 predictors and on the 23 raw ones. Depth 3 was selected on these folds without nesting, so its gain over depth 1 is if anything an upper estimate; rounds re-tuned by inner 3-fold CV inside each outer fold (rounds per fold under the AUC). Source: `results/ablation_depth.csv`.

	29 predictors (23 raw + 6 engineered)		23 raw predictors	
Model	300 rounds	Rounds re-tuned (rounds per fold)	300 rounds	Rounds re-tuned (rounds per fold)
Plain logit		0.7602		0.7230
XGBoost depth 1 (additive)	0.7756	0.7827 1,200/1,200/2,400/1,200/1,200	0.7705	0.7766 1,200/1,200/2,400/2,400/1,200
XGBoost depth 2	0.7840	0.7851 600/600/600/600/600	0.7787	0.7806 600/600/600/600/600
XGBoost depth 3 (frozen)	0.7863	0.7863 300/300/300/300/300	0.7819	0.7819 300/300/300/300/300
XGBoost depth 4	0.7859	0.7859 300/300/300/300/300	0.7828	0.7828 300/300/300/300/300

Fold-paired differences in CV AUC (29 predictors unless stated): total gap, depth 3 − logit $+0.0261$ [$+0.0145$, $+0.0376$], 5/5; additive booster, depth 1 (re-tuned) − logit $+0.0225$ [$+0.0111$, $+0.0338$], 5/5; interactions, depth 3 − depth 1, both re-tuned $+0.0036$ [$+0.0022$, $+0.0050$], 5/5; the same at 300 rounds $+0.0107$ [$+0.0071$, $+0.0144$], 5/5 (depth 1 is under-fitted at 300 rounds); on the 23 raw columns, depth 3 − depth 1 re-tuned $+0.0053$ [$+0.0031$, $+0.0075$], 5/5; difference-in-differences (raw minus engineered interaction gain) $+0.0017$ [-0.0014, $+0.0048$], 5/5. Value of the six engineered features: to the logit $+0.0372$ [$+0.0309$, $+0.0434$], 5/5, to XGBoost at depth 3 $+0.0044$ [$+0.0011$, $+0.0077$], 5/5. The additive booster recovers 86 % of the gap; the partial-dependence curves show where its one-variable shape lies, but no share per variable was measured.

Table A26: Recent versus older repayment history (post-freeze, training CV only, never test-scored): CV AUC of XGBoost (frozen settings) and the plain logit as months of history are added, number of predictors in brackets. Raw months: the monthly status, bill and payment of each month in the window. Six-month summaries: `UTIL_1`, `UTIL_MEAN`, `PAY_RATIO_MEAN`, `N_MONTHS_DELAYED`, `MAX_DELAY`, `BILL_TREND`, always computed over all six months; window summaries recompute them over the months in the window only. H6 with either kind of summary is the frozen 29-predictor model. Source: `results/ablation_history.csv`.

	XGBoost			Plain logit		
History window	Raw months	+ six-month summaries	+ window summaries	Raw months	+ six-month summaries	+ window summaries
H0: limit + demographics	0.6299 (5)	0.7742 (11)	—	0.6277 (5)	0.7492 (11)	—
H1: + September	0.7688 (8)	0.7866 (14)	0.7691 (12)	0.7165 (8)	0.7463 (14)	0.7478 (12)
H3: + July–September	0.7795 (14)	0.7864 (20)	0.7840 (20)	0.7241 (14)	0.7558 (20)	0.7551 (20)
H6: + April–September	0.7819 (23)	0.7863 (29)	0.7863 (29)	0.7230 (23)	0.7602 (29)	0.7602 (29)

Fold-paired differences, XGBoost, raw months: H1 − H0 $+0.1389$ [$+0.1090$, $+0.1689$], 5/5; H3 − H1 $+0.0108$ [$+0.0076$, $+0.0139$], 5/5; H6 − H3 $+0.0024$ [-0.0004, $+0.0052$], 5/5; all older months together, H6 − H1 $+0.0131$ [$+0.0075$, $+0.0188$], 5/5. With the six-month summaries, H6 − H1 -0.0003 [-0.0028, $+0.0022$], 2/5: once the summaries are in, the older raw months add nothing measurable for XGBoost (for the logit they still add $+0.0140$ [$+0.0088$, $+0.0192$], 5/5). Six-month minus September-window summaries at H1 $+0.0175$ [$+0.0128$, $+0.0221$], 5/5: the older history enters through the summaries. H1 adds September's status, bill and payment together, not the status alone. Code 1 occurs almost only in September (Table A21), so part of what H1 adds is that month's coding.

Table A27: Incremental value of SEX, AGE, MARRIAGE and EDUCATION (post-freeze, training CV only, never test-scored): the frozen XGBoost and plain logit refitted without the four columns on identical folds and settings. Each difference is with minus without (mean, k/5 with the difference positive, Nadeau–Bengio interval); for Brier and cost a negative difference favours the model with the demographics. The 5:1 cut-off is chosen on the other four folds (cross-fitted); a fold holds about 4,195 clients. Source: `results/ablation_demographics.csv`.

Quantity	XGBoost	Plain logit
CV AUC with SEX, EDUCATION, MARRIAGE, AGE (29 predictors)	0.7863	0.7602
CV AUC without them (25 predictors)	0.7842	0.7565
ΔAUC, with − without	$+0.0021$, 5/5	$+0.0037$, 5/5
	[$+0.0002$, $+0.0041$]	[-0.0004, $+0.0079$]
pooled-OOF bootstrap 95 % interval	[$+0.0009$, $+0.0034$]	[$+0.0016$, $+0.0060$]
ΔAP	$+0.0011$, 2/5	$+0.0028$, 4/5
	[-0.0051, $+0.0074$]	[-0.0022, $+0.0079$]
ΔBrier	-0.0004, 1/5	-0.0005, 0/5
	[-0.0010, $+0.0002$]	[-0.0010, $+0.0000$]
Δ cross-fitted 5:1 cost per fold	-62.2, 0/5	-9.8, 2/5
	[-103.2, -21.2]	[-59.5, $+39.9$]
Δ recall at the 5:1 cut-off	$+0.036$, 4/5	-0.051, 0/5
	[-0.044, $+0.115$]	[-0.117, $+0.014$]

The no-demographics out-of-fold vectors (`ablation_oof_*_nodemo.npy`) are identical to those of the fairness check (Table A28).

Table A28: Group outcomes at the 5:1 cut-off (training out-of-fold scores, post-freeze diagnostic, never test-scored). This is a diagnostic on training out-of-fold scores, not a compliance assessment. PD − obs.: mean predicted probability of default minus the observed default rate, in percentage points; flagged: share of the group at or above the cut-off; FPR: false-alarm rate, the share of the group's non-defaulters flagged. 'No demogr.': XGBoost refitted without SEX, AGE, MARRIAGE and EDUCATION (Table A27). 'Others/unknown' education pools documented 'others' with the undocumented codes 0, 5 and 6 — a coding artefact, not an education effect. Sources: `results/fairness_groups.csv`, `fairness_summary.json`, `fairness_proxies.csv`.

			XGBoost (frozen)			Logit (frozen)		XGBoost, no demogr.	
Group	n	Default (%)	PD − obs. (pp)	Flagged (%)	FPR (%)	Flagged (%)	FPR (%)	Flagged (%)	PD − obs. (pp)
All training clients	20,975	22.1	−0.03	49.8	41.1	45.2	37.0	49.9	−0.00
Sex									
male	8,342	24.5	−0.48	54.7	45.7	53.8	45.5	52.8	−1.12
female (ref.)	12,633	20.6	+0.27	46.5	38.2	39.5	31.6	48.0	+0.73
Age band									
<30	6,676	22.8	+0.04	51.2	42.2	43.8	34.7	54.0	+0.88
30–39 (ref.)	7,926	20.5	+0.03	46.0	37.7	41.8	34.1	46.0	+0.07
40–49	4,490	22.9	−0.33	51.4	42.6	48.9	40.9	49.2	−1.07
50+	1,883	25.0	+0.22	57.0	49.0	55.5	48.4	53.5	−0.89
Marital status									
married	9,536	23.7	−0.50	52.6	43.8	52.1	44.1	49.4	−1.39
single (ref.)	11,173	20.8	+0.33	47.0	38.6	39.2	31.0	50.1	+1.15
others	266	23.3	+1.96	64.3	56.4	48.5	41.7	59.0	+1.18
Education (recoded)									
graduate school	7,321	19.0	+0.16	43.4	35.5	36.3	28.9	44.7	+0.41
university (ref.)	9,831	23.9	−0.34	53.3	44.5	50.1	41.7	52.0	−0.61
high school	3,480	25.2	+0.22	57.2	48.0	53.3	45.1	56.3	−0.05
others/unknown	343	6.7	+2.36	10.8	10.0	10.8	10.0	35.9	+8.97

Cut-offs: the frozen 5:1 cut-off 0.15 for both frozen models (chosen on their own training out-of-fold scores); the demographics-free refits use the same cost rule on their own out-of-fold scores (0.15 XGBoost, 0.14 logit). Flag-rate ratios to the reference group: XGBoost men/women 1.18 against a default-rate ratio of 1.19; plain logit 1.36 (false-alarm ratio 1.44) and married/single 1.33. Scoring every client with the reference values of the four attributes ('neutralised') gives 1.08 / 1.00 (sex / married) for the logit and 1.09 / 0.99 for XGBoost; the demographics-free XGBoost gives 1.10 / 0.99, but its under-30 ratio rises from 1.11 to 1.17. Proxies: the 25 non-demographic predictors recover female 0.60, age under 30 0.65, single 0.56, graduate school 0.69 (out-of-fold AUC), with `LIMIT_BAL` the leading proxy each time. PD density near the cut-off: 5.1 % of clients per 0.01 of PD for the logit against 3.5 % for XGBoost. Largest |mean PD − observed| of the frozen XGBoost in groups of 1,000 or more clients: 0.50 pp.

Table A29: Monotone constraints re-specified (post-freeze, training CV only). The respecified model is a training-CV check only and was never scored on the test set; its guarantee holds from revolving credit upwards. Part A: XGBoost with the frozen settings under six constraint sets, on the numeric coding of the repayment codes or recoded (months late plus indicators for -2 and -1, as in Table A22); $+$ / $-$ = number of columns constrained to raise / lower the risk; 'utilisation freed' keeps the frozen signs except on the two utilisation columns; SD over the five folds, ddof 1. Part B: wrong-direction steps of the predicted probability. Source: `results/monotone_respec_cv.json`, `monotone_check_consistent.csv`.

Variant	$+$ / $-$	CV AUC $\pm$ SD	CV AP	Brier	Δ vs free (frozen)	Δ vs monotone (frozen)
Numeric coding (as frozen)						
Free (as frozen)	—	0.7863 ± 0.0076	0.5540	0.1344	—	$+0.0019$, 5/5 $[+0.0013, +0.0025]$
Monotone (frozen)	11 / 2	0.7844 ± 0.0077	0.5459	0.1351	-0.0019, 0/5 $[-0.0025, -0.0013]$	—
Recoded: months late + flags for -2, -1						
Free	—	0.7866 ± 0.0074	0.5542	0.1344	$+0.0003$, 4/5 $[-0.0003, +0.0009]$	$+0.0023$, 5/5 $[+0.0014, +0.0031]$
Frozen signs	11 / 2	0.7850 ± 0.0074	0.5464	0.1349	-0.0013, 0/5 $[-0.0026, +0.0000]$	$+0.0006$, 4/5 $[-0.0005, +0.0018]$
Utilisation freed	9 / 2	0.7854 ± 0.0072	0.5512	0.1347	-0.0009, 1/5 $[-0.0021, +0.0003]$	$+0.0010$, 5/5 $[-0.0003, +0.0023]$
Respecified	8 / 1	0.7858 ± 0.0075	0.5526	0.1346	-0.0005, 1/5 $[-0.0015, +0.0005]$	$+0.0014$, 5/5 $[+0.0007, +0.0021]$

Part B: wrong-direction steps over 200 random training profiles, consistent sweep unless stated

Variant	All steps, as frozen	All steps, consistent	Non-delinquent	Entry into arrears	Between delay levels (thin)	Cross-state pairs (largest fall)
Free (as frozen)	576	573	213	$3 \to 0$	360 (360)	413 (0.062)
Monotone (frozen)	0	0	0	$0 \to 0$	0 (0)	0 (0.000)
Free	577	564	199	$13 \to 1$	364 (364)	538 (0.076)
Respecified	199	199	199	$0 \to 0$	0 (0)	35 (0.024)

Respecified constraints: $+$ on `PAY_1_DEL`, `PAY_2_DEL`, `PAY_3_DEL`, `PAY_4_DEL`, `PAY_5_DEL`, `PAY_6_DEL`, `N_MONTHS_DELAYED`, `MAX_DELAY`; $-$ on `LIMIT_BAL`; free: the -2 / -1 indicators and `BILL_TREND`, `PAY_RATIO_MEAN`, `UTIL_1`, `UTIL_MEAN`. Ladder, fold-paired AUC: price of the frozen constraint -0.0019 $[-0.0025, -0.0013]$, 0/5; price of the respecified one against the free recoded booster -0.0008 $[-0.0016, -0.0000]$, 0/5; respecified $-$ frozen monotone $+0.0014$ $[+0.0007, +0.0021]$, 5/5. Part B: each profile's `PAY_1` is swept over the codes seen in training, 2,000 adjacent steps in all (400 between non-delinquent codes, 200 entries into arrears, 1,400 between delay levels of which 1,000 in thin cells, where either code has fewer than 100 training clients in that month; thin-cell violations in brackets). 'As frozen' holds `MAX_DELAY` and `N_MONTHS_DELAYED` fixed while `PAY_1` moves, as in Figure A28; 'consistent' recomputes them. The respecified constraint leaves the non-delinquent steps free by design. Cross-state pairs: for every month, each code -2 or -1 against each observed $k \geq 1$, 17,600 pairs; a pair violates when the probability at k is lower. Other variants: frozen signs 56 (largest 0.020); utilisation freed 64 (largest 0.028).

Table A30: No-parameter arrears rules against the models on the training rows at a 5:1 cost ratio (post-freeze, never test-scored). Cost is $5\,\mathrm{FN} + \mathrm{FP}$ per 1,000 training clients; for the two models it is cross-fitted (the cut-off chosen on the other four folds). Savings are rule minus model per 1,000 clients over the five folds (k/5 = folds in which the model is cheaper). The last two columns give the recall of XGBoost and of the logit when they flag the same share of clients as the rule. Source: `results/simple_rule_benchmark.csv`.

Rule or model	Flagged (%)	Recall	Precision	False alarms per catch	Cost per 1,000	XGBoost saving k/5 [NB]	Logit saving k/5 [NB]	Recall at the same share: XGB	Recall at the same share: Logit
`PAY_1` ≥ 1	22.6	0.512	0.500	1.00	653.7	+113, 5/5 [+85, +142]	+73, 5/5 [+43, +103]	0.550	0.534
`PAY_1` ≥ 2	10.5	0.326	0.684	0.46	779.2	+239, 5/5 [+216, +262]	+199, 5/5 [+160, +237]	0.327	0.300
`MAX_DELAY` ≥ 2	28.0	0.583	0.461	1.17	612.0	+72, 5/5 [+42, +102]	+31, 5/5 [+14, +49]	0.616	0.598
`N_MONTHS_DELAYED` ≥ 1 (any delay in six months)	33.6	0.645	0.425	1.35	585.6	+45, 5/5 [+25, +66]	+5, 4/5 [−4, +14]	0.667	0.650
Flag everybody	100.0	1.000	0.221	3.52	778.7	+238, 5/5 [+217, +260]	+198, 5/5 [+160, +236]	—	—
Flag nobody	0.0	0.000	—	—	1106.3	+566, 5/5 [+545, +587]	+526, 5/5 [+488, +564]	—	—
Logistic (plain) at 0.15	45.2	0.741	0.363	1.76	580.5	—	—	0.766	0.741
XGBoost at 0.15	49.8	0.802	0.357	1.80	540.3	—	—	0.802	0.770

The rules have no fitted parameter, so the training rows are a fair test for them; they were not scored on the test set. Any payment delay in the six months (`N_MONTHS_DELAYED` ≥ 1) was the cheapest of the four candidate rules on these same rows, a mild selection effect in its favour; XGBoost still costs 7.7 % less than it. The models' cut-off (0.15) was chosen on pooled training out-of-fold scores.

Table A31: Cheapest of the eleven headline models at each false-negative to false-positive cost ratio r on pooled training out-of-fold scores (reporting only; no cost winner is named, and nothing here was scored on the test set). Top: the cheapest model with its cut-off (chosen on the same scores), the share of clients it flags, its recall and cost per 1,000 clients, the runner-up with its cost margin in per cent, and the cost spread of the top three in per cent. Bottom: the cheapest model when every fold's cut-off is chosen on the other four (cross-fitted), the runner-up minus cheapest cost per 1,000 over the folds with the folds in which the cheapest is cheaper, the ranks of the plain logit and the pruned tree (gap to the cheapest, %), and the best blanket rule with the cheapest model's saving against it (%). Source: `results/preferred_model_by_ratio_oof.csv`.

r	$1/(1+r)$	Cheapest (cut-off)	Flagged (%)	Recall	Cost per 1,000	Runner-up (margin, %)	Top-3 spread (%)
1:1	0.500	GBM (0.49)	12.0	0.367	179.5	XGB (0.66)	0.69
2:1	0.333	RF (0.33)	21.6	0.537	302.4	XGB (0.60)	0.98
3:1	0.250	XGB (0.27)	25.7	0.591	397.8	GBM (0.22)	0.80
5:1	0.167	XGB (0.15)	49.8	0.802	538.8	GBM (0.70)	1.03
7:1	0.125	XGB (0.13)	57.2	0.852	613.0	RF (0.35)	0.87
10:1	0.091	RF (0.09)	74.9	0.938	679.9	GBM (0.55)	0.90
15:1	0.062	RF (0.06)	86.1	0.973	735.7	GBM (0.58)	0.62
20:1	0.048	KNN (0.03)	96.0	0.996	759.1	XGB (0.13)	0.64

r	Cheapest, cross-fitted	Runner-up − cheapest per 1,000 [NB]	Cheapest in k/5 folds	Logit rank (gap, %)	Tree rank (gap, %)	Best blanket rule (saving, %)
1:1	GBM	$+3.0$ $[-1.0, +6.9]$	5/5	9 (7.8)	4 (1.3)	flag nobody (18.9)
2:1	RF	$+1.0$ $[-7.8, +9.8]$	2/5	8 (3.9)	4 (2.6)	flag nobody (31.7)
3:1	GBM (differs)	-0.6 $[-6.7, +5.5]$	3/5	10 (4.0)	5 (3.7)	flag nobody (40.1)
5:1	XGB	$+2.3$ $[-12.6, +17.2]$	3/5	7 (6.7)	6 (6.3)	flag everybody (30.8)
7:1	XGB	$+2.1$ $[-16.5, +20.8]$	3/5	7 (9.4)	6 (4.8)	flag everybody (21.3)
10:1	RF	$+5.1$ $[-10.6, +20.7]$	4/5	8 (7.6)	4 (1.9)	flag everybody (12.7)
15:1	RF	$+2.7$ $[-21.6, +26.9]$	3/5	8 (5.1)	6 (3.6)	flag everybody (5.5)
20:1	KNN	$+5.6$ $[-33.8, +45.1]$	2/5	9 (2.9)	6 (2.4)	flag everybody (2.5)

XGB = XGBoost, GBM = gradient boosting, RF = random forest, KNN = K-nearest neighbours ($K = 101$).

Table A32: Who is caught in the riskiest 10, 20 and 30 % of the training clients (training out-of-fold scores of the frozen configurations; post-freeze, never test-scored): the share of all defaulters captured and the precision of the list, for the eleven models, three arrears counts used as scores and a random list. XGB − row: XGBoost's capture minus the row's at 10 %, in percentage points, over the five folds. Source: `results/topk_capture_oof.csv`.

	Riskiest 10 %			Riskiest 20 %		Riskiest 30 %	
List ranked by	Captured (%)	Precision (%)	XGB − row k/5 [NB]	Captured (%)	Precision (%)	Captured (%)	Precision (%)
XGBoost	31.5	69.6	—	50.7	56.1	63.3	46.7
Gradient boosting	31.5	69.6	−0.1, 1/5 [−0.3, +0.2]	50.8	56.2	63.1	46.5
Random forest	31.0	68.6	+0.4, 5/5 [−0.2, +1.1]	51.1	56.5	63.3	46.7
MLP	30.8	68.1	+0.8, 5/5 [−0.8, +2.5]	49.7	55.0	62.1	45.8
KNN ($K = 101$)	29.2	64.7	+2.3, 5/5 [+0.7, +3.8]	49.4	54.6	61.9	45.7
Pruned tree (17 leaves)	31.5	69.6	+0.2, 4/5 [−0.3, +0.8]	49.7	55.0	61.3	45.2
SVM (RBF)	27.4	60.5	+4.1, 5/5 [+1.8, +6.5]	49.5	54.8	61.5	45.3
Logistic (plain)	28.7	63.4	+2.8, 5/5 [+0.8, +4.8]	49.4	54.6	61.7	45.5
Logistic L1	28.7	63.4	+2.8, 5/5 [+0.8, +4.8]	49.4	54.6	61.7	45.5
LDA	28.8	63.7	+2.6, 5/5 [+0.7, +4.5]	49.0	54.3	61.5	45.3
Gaussian NB	28.2	62.4	+3.3, 5/5 [+1.4, +5.3]	47.2	52.2	60.5	44.6
Rule score: `N_MONTHS_DELAYED`	27.8	61.5	+3.7, 5/5 [+1.9, +5.5]	46.3	51.2	59.7	44.0
Rule score: `MAX_DELAY`	23.1	51.2	+8.4, 5/5 [+6.5, +10.3]	42.7	47.2	60.5	44.6
Rule score: `PAY_1`	30.9	68.5	+0.6, 5/5 [+0.0, +1.2]	47.1	52.1	55.4	40.9
Random list	10.0	22.1	—	20.0	22.1	30.0	22.1

Scores with ties at the list boundary (the rule scores, KNN and the tree) are counted at their expected value over the tied clients; tied clients at the boundary for the rule scores, 10/20/30 %: `MAX_DELAY` 5,046/5,046/1,162; `N_MONTHS_DELAYED` 808/3,081/3,081; `PAY_1` 1,871/2,540/10,380. A perfect list would capture 45.2, 90.4 and 100.0 % of the defaulters.

Table A33: R cross-check on the same five training folds: the course's R models refitted on `data/processed/cv_folds.csv` with the frozen settings; 13 of 15 mirrors within 0.005 of the Python CV AUC, Spearman 0.99 between the two columns over the mirrors (`code/r_crosscheck/README.md`); the test file is never opened. [d]Diagnostic refit with the Python setting; [c]contrast with the lab defaults (neither counts as a mirror). Source: `results/r_crosscheck.csv`, whose `notes` column gives each explanation in full.

R model	R CV AUC $\pm$ SD	Python CV AUC	R $-$ Python	Folds R higher	Note
glm (logistic)	0.7602 ± 0.0112	0.7602	$+0.0000$	0/5	same likelihood; IRLS against lbfgs
cv.glmnet ridge	0.7597 ± 0.0113	0.7602	-0.0006	1/5	λ by inner 10-fold deviance; glmnet also scales the dummies
cv.glmnet lasso	0.7595 ± 0.0112	0.7602	-0.0007	1/5	as ridge; both penalties end up tiny
MASS::lda	0.7572 ± 0.0122	0.7572	$+0.0000$	0/5	same model
MASS::qda	0.7394 ± 0.0108	0.7511	-0.0118	0/5	no regularisation: reproduces the Python `reg_param` = 0 grid point
QDA reg 0.5 (by hand)[d]	0.7511 ± 0.0104	0.7511	$+0.0000$	4/5	QDA with $r = 0.5$ written by hand: matches the frozen Python model
e1071::naiveBayes	0.7353 ± 0.0125	0.7356	-0.0003	1/5	Python adds `var_smoothing` to every variance
class::knn K=101	0.7700 ± 0.0097	0.7700	-0.0000	3/5	same K and inputs; `class::knn` keeps all distance ties
class::knn K=5	0.7061 ± 0.0072	0.7061	-0.0001	1/5	mirrors the round-1 $K = 5$ variant
tree + CV pruning	0.7669 ± 0.0068	0.7689	-0.0020	2/5	deviance tree, pruned by inner CV; Python prunes by `ccp_alpha`
tree lab defaults (prune.misclass)[c]	0.7347 ± 0.0073	0.7689	-0.0342	0/5	lab defaults stop at a few leaves
randomForest	0.7743 ± 0.0090	0.7849	-0.0106	0/5	hard votes and smaller leaves than scikit-learn's soft votes, leaf ≥ 20
randomForest soft votes[d]	0.7781 ± 0.0083	0.7849	-0.0068	0/5	the same R trees averaged as leaf shares
randomForest soft votes, nodesize 80[d]	0.7828 ± 0.0087	0.7849	-0.0021	0/5	soft votes with leaves about as large as scikit-learn's
gbm 200 trees	0.7831 ± 0.0091	0.7850	-0.0018	1/5	`interaction.depth` 3 = 4 leaves; scikit-learn depth 3 allows 8
gbm 300 trees	0.7844 ± 0.0088	0.7850	-0.0005	2/5	as above, read at 300 trees
xgboost	0.7862 ± 0.0080	0.7863	-0.0001	3/5	same library and parameters; only the subsampling draws differ
e1071::svm radial	0.7635 ± 0.0127	0.7635	$+0.0000$	3/5	libsvm in both, same C, γ and weights
nnet	0.7776 ± 0.0065	0.7762	$+0.0014$	3/5	one sigmoid layer (BFGS) against two ReLU layers (Adam)

A.3 Post-freeze checks (training rows only)

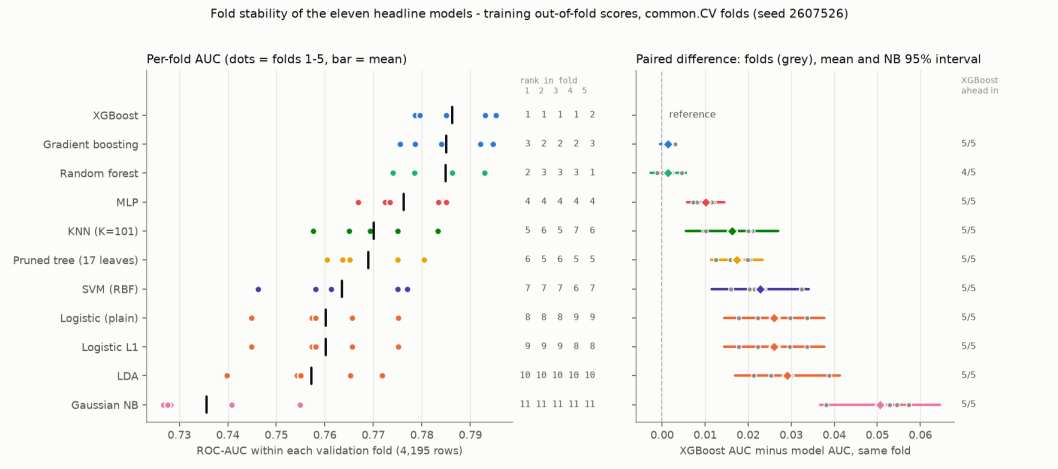


Figure A1: Fold stability of the eleven headline configurations (post-freeze check on the training out-of-fold scores of the frozen configurations; nothing re-fitted, nothing test-scored). Left: AUC in each of the five validation folds (dots), the mean (bar) and the rank in each fold. Right: XGBoost minus each model in the same fold (grey dots), mean and Nadeau–Bengio 95 % interval, with the number of folds XGBoost is ahead (Table A23).

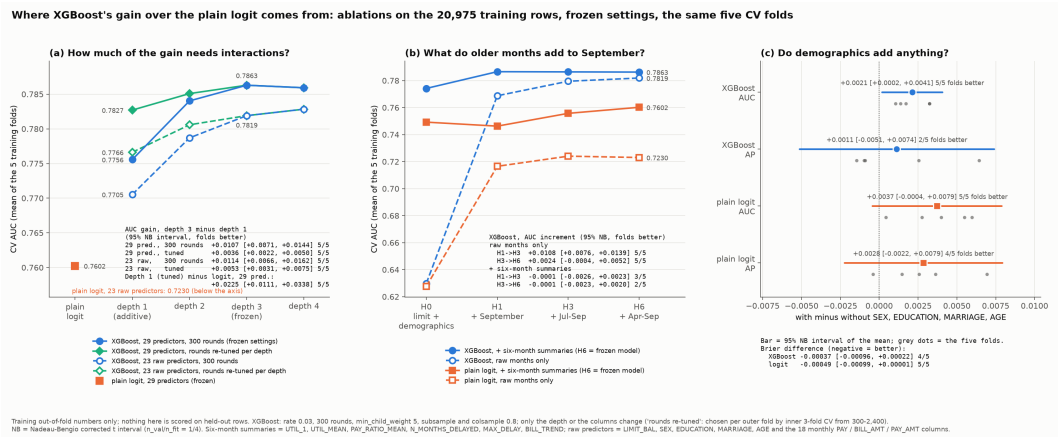


Figure A2: Where XGBoost's gain over the plain logit comes from: post-freeze ablations, training five-fold CV with the frozen settings, never test-scored. (a) Interaction depth (Table A25); (b) recent against older repayment history (Table A26); (c) the incremental value of SEX, EDUCATION, MARRIAGE and AGE (Table A27).

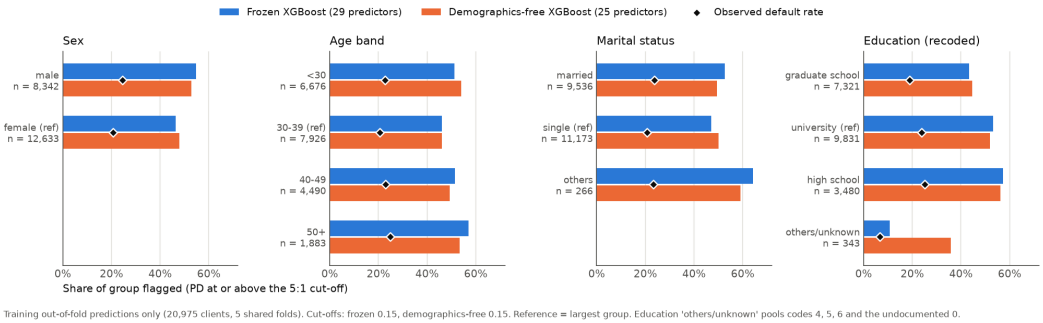


Figure A3: Share of each group flagged at the 5:1 cut-off by the frozen XGBoost and by its demographics-free refit, with the observed default rate (diamonds). Training out-of-fold scores; a post-freeze diagnostic, not a compliance assessment (Table A28).

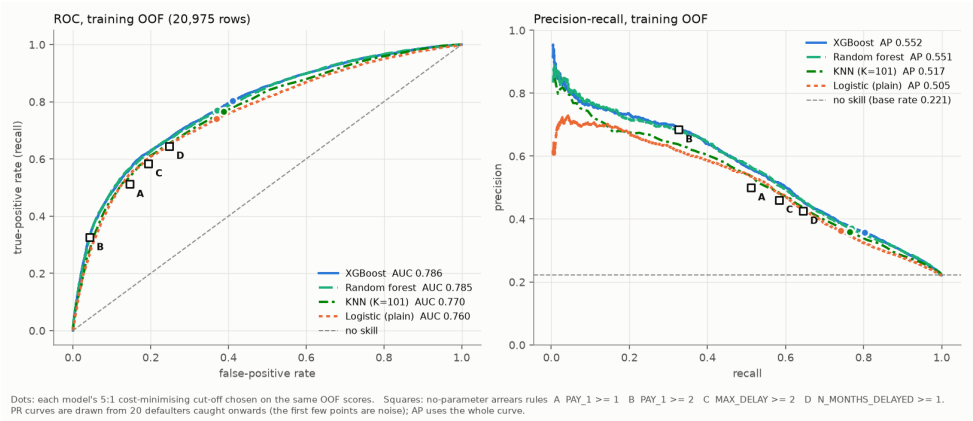


Figure A4: ROC and precision–recall curves on the training out-of-fold scores of four frozen configurations, with each model's 5:1 cut-off (dots) and the four no-parameter arrears rules A–D (squares; Table A30). Training rows only; the test-set curves of the frozen configurations are Figures A13 and A14.

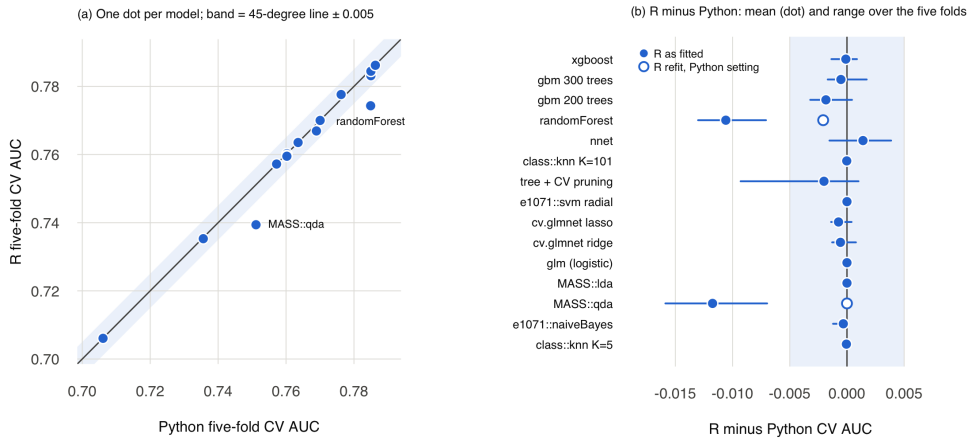


Figure A5: R cross-check on the same five training folds: (a) R against Python CV AUC, one dot per model, band = 45-degree line $\pm$ 0.005; (b) R minus Python, mean and range over the five folds; open circles are diagnostic R refits with the Python setting (Table A33). The test file is never opened.

A.4 Exploratory analysis

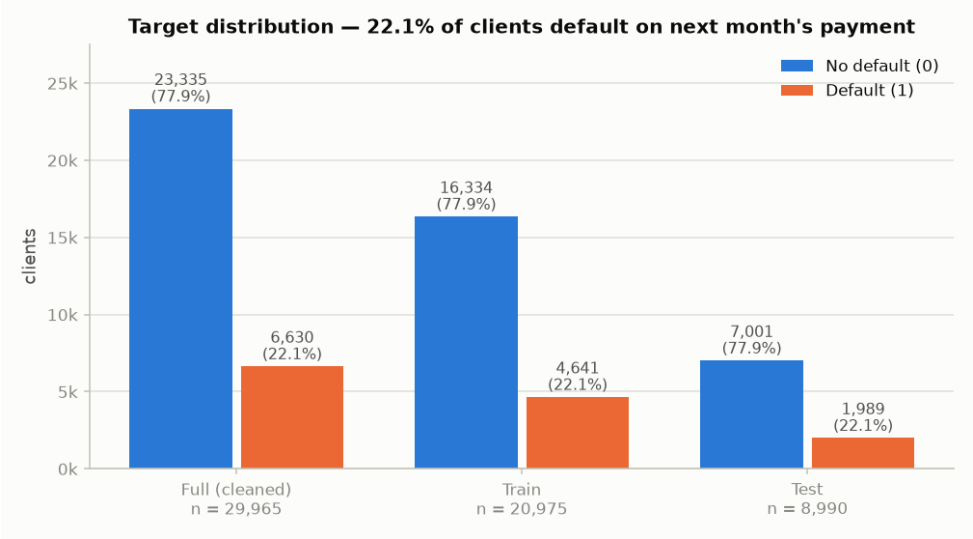


Figure A6: Class balance of the cleaned sample and of the stratified training and test partitions.

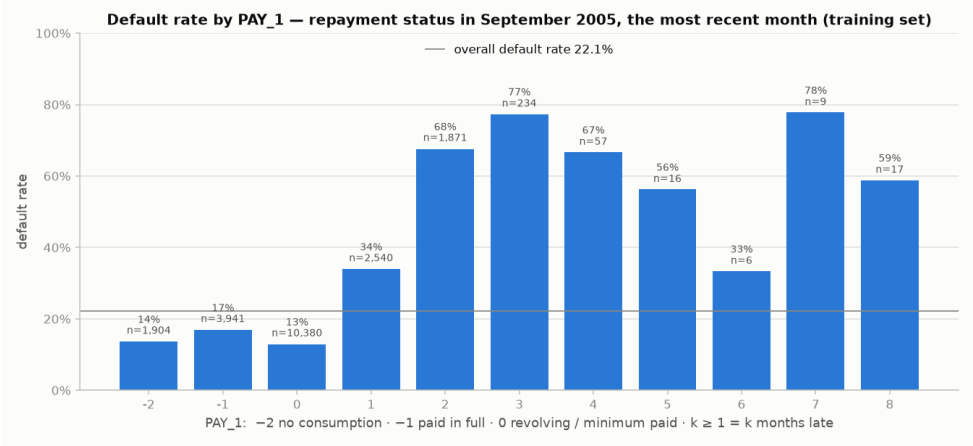


Figure A7: Default rate by the most recent repayment status, training partition.

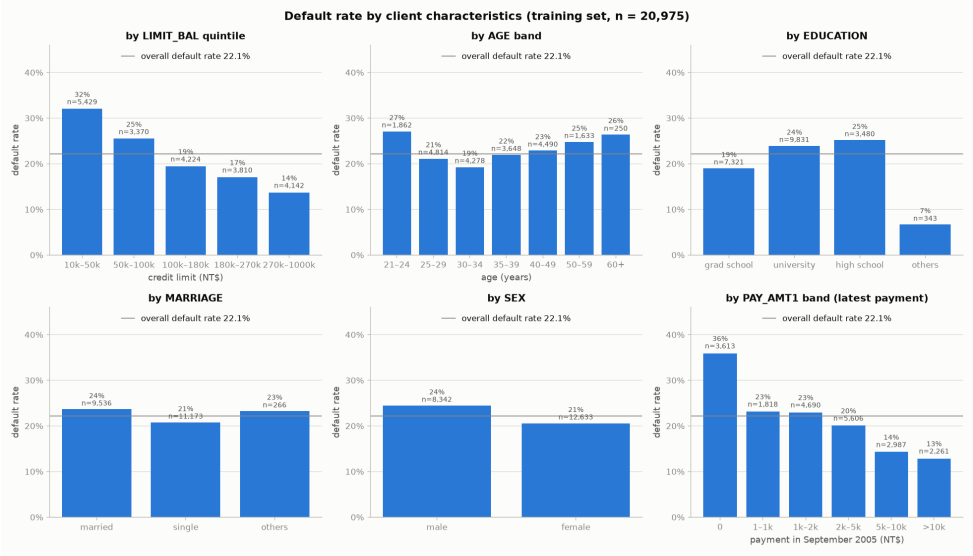


Figure A8: Training-partition default rate by credit-limit quintile, age band, education, marriage and sex.

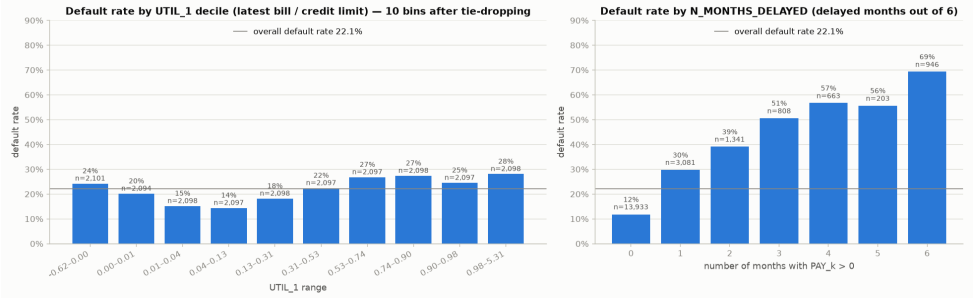


Figure A9: Default rate by utilisation decile and by number of months in arrears, training partition.

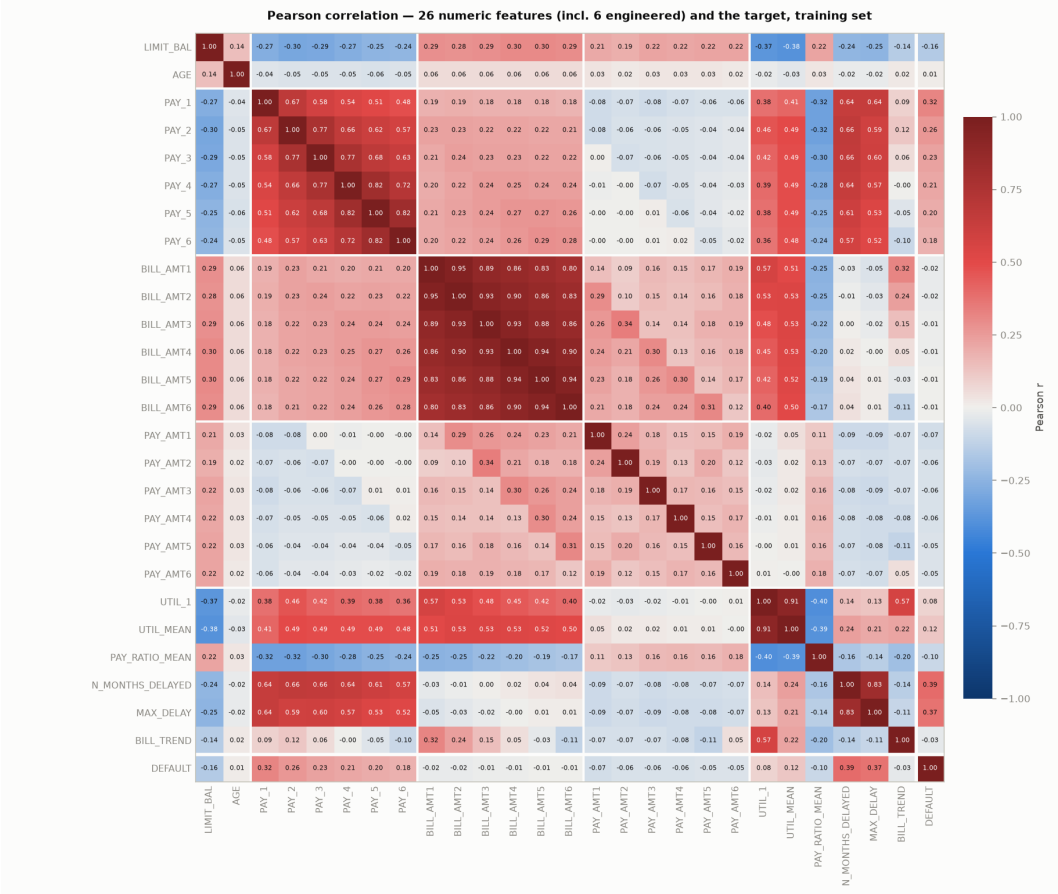


Figure A10: Correlation heat-map of the 29 predictors and the target.

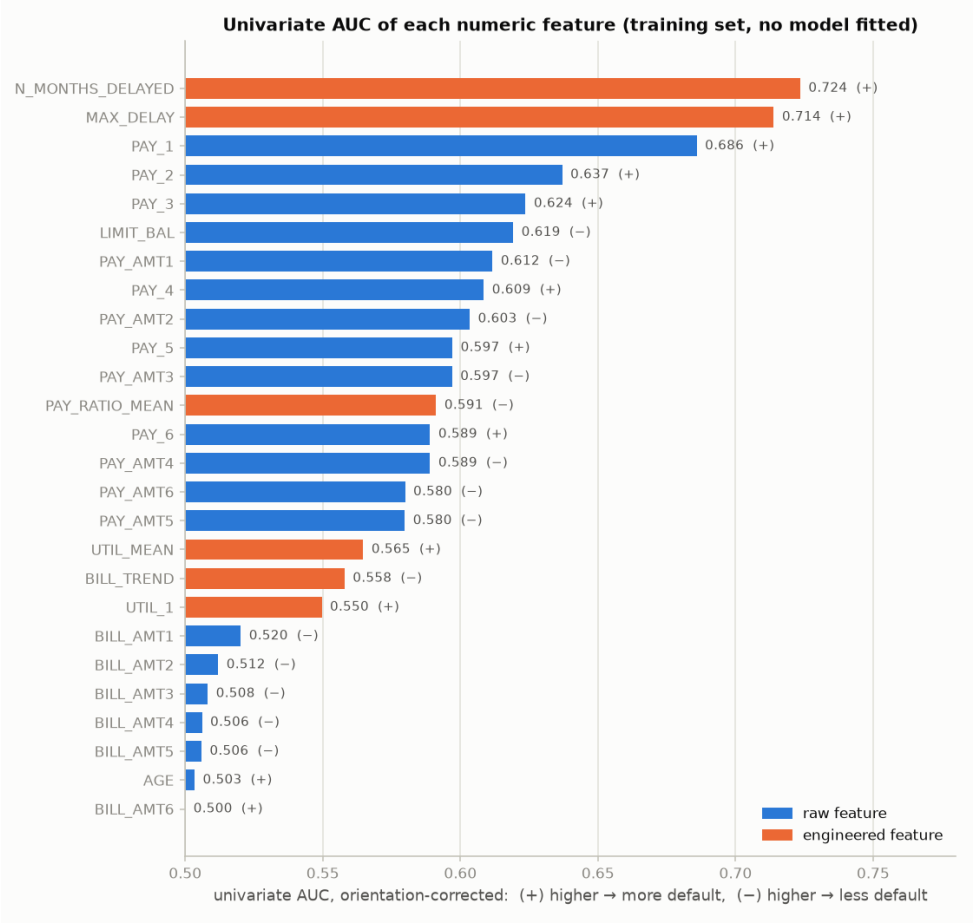


Figure A11: Univariate strength of every predictor, as an oriented AUC.

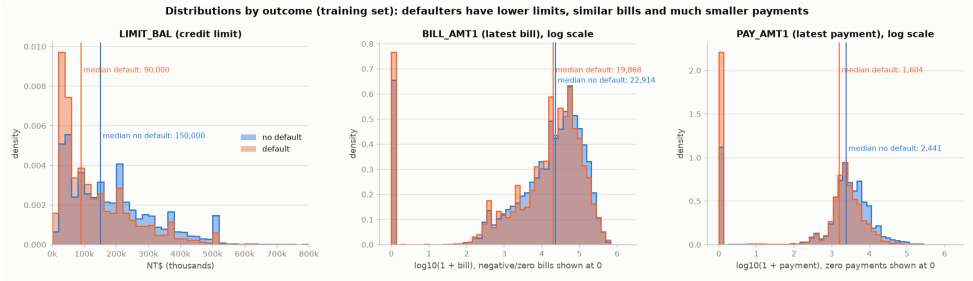


Figure A12: Distributions of the main numeric features by class, training partition.

A.5 Model comparison

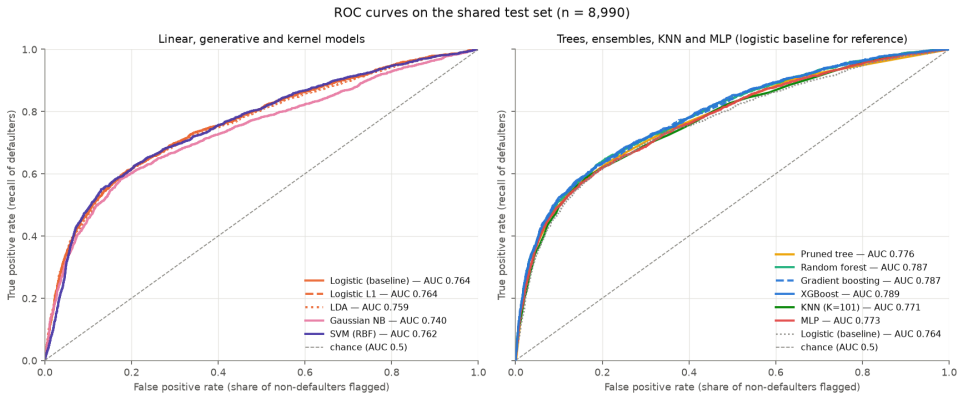


Figure A13: ROC curves of the headline models. Test set, frozen configurations; reporting only.

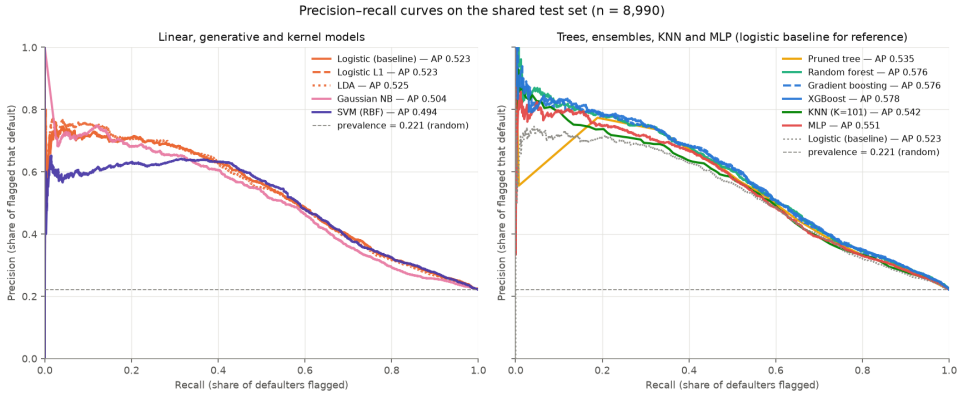


Figure A14: Precision–recall curves of the headline models; the horizontal line is the prevalence. Test set, frozen configurations; reporting only.

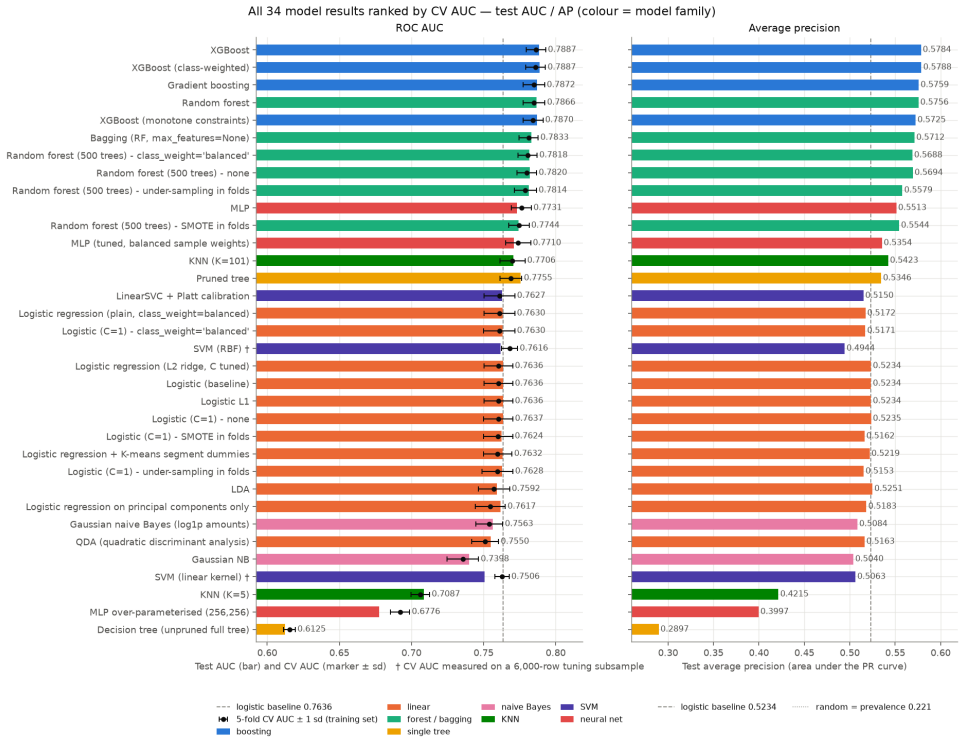


Figure A15: Test AUC of every model variant fitted, ordered by CV AUC, with the benchmark marked.

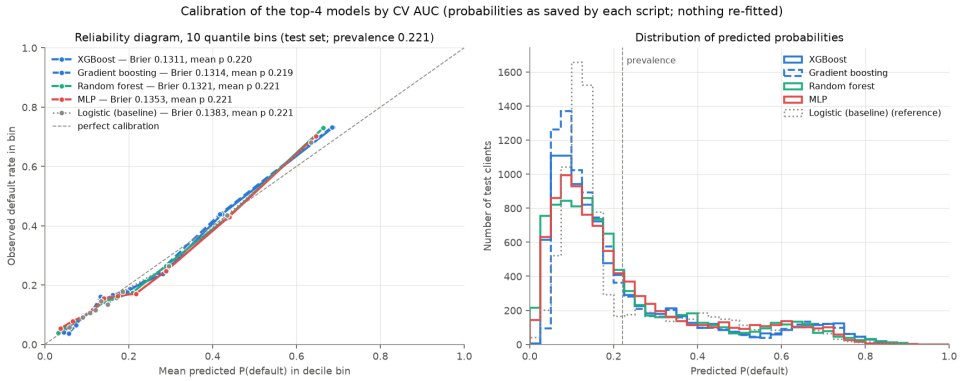


Figure A16: Calibration curves of the four models with the highest CV AUC: predicted against observed default rate by decile of score.

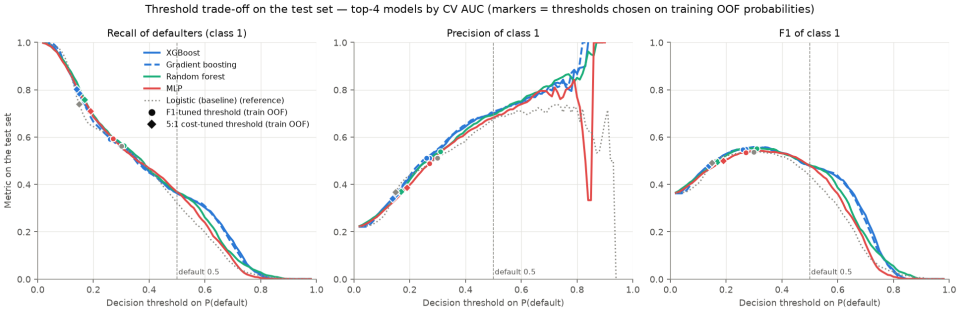


Figure A17: Recall, precision and F1 against the decision threshold.

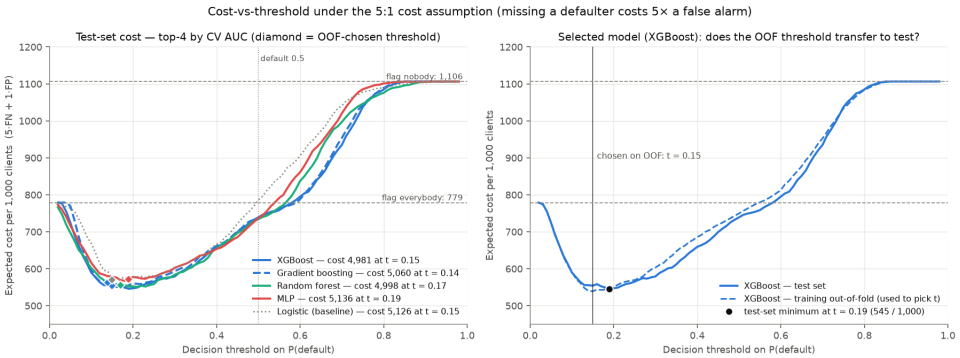


Figure A18: Expected cost against the decision threshold at a 5:1 ratio.

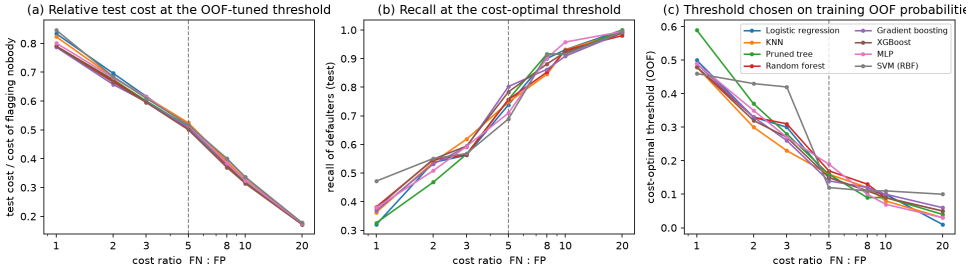


Figure A19: Cost-minimising threshold and cost as the cost ratio varies from 1:1 to 20:1.

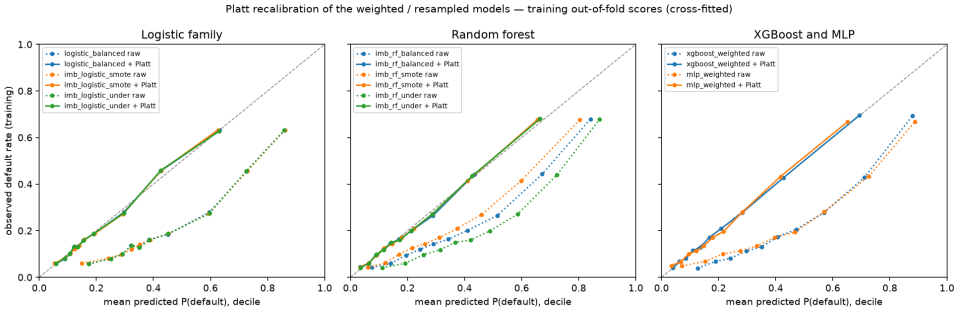


Figure A20: Reliability of the nine weighted or resampled models before and after Platt recalibration, on training out-of-fold scores.

A.6 Interpretation

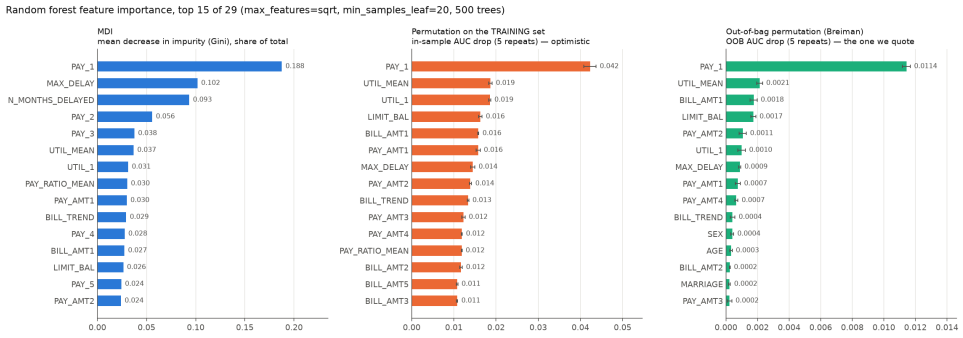


Figure A21: Random forest importances: impurity decrease (MDI), training-set permutation and out-of-bag permutation, side by side. MDI favours columns with many split points.

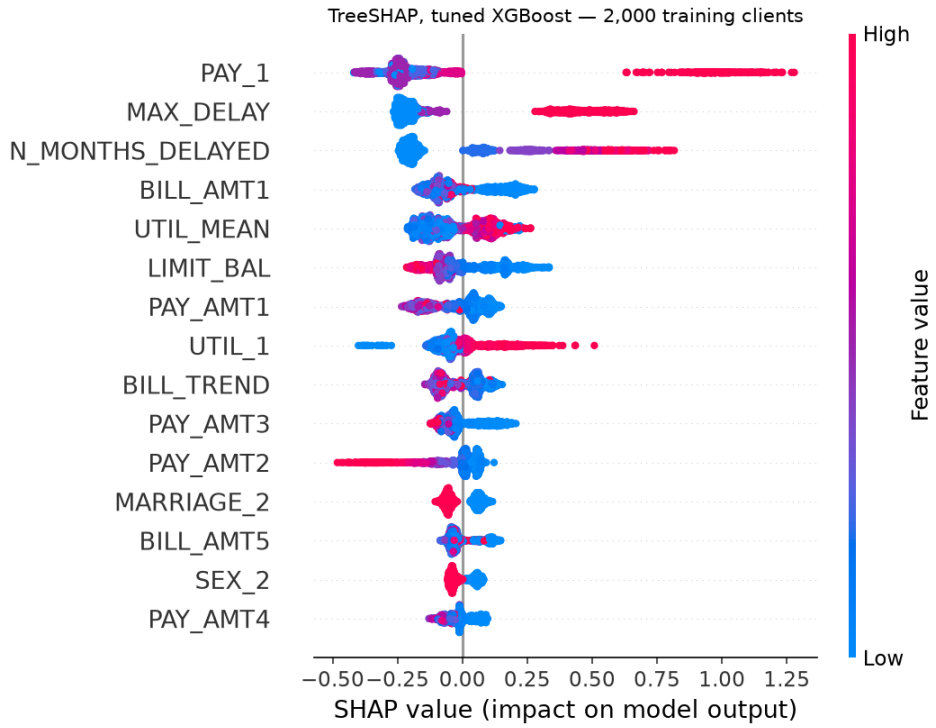


Figure A22: TreeSHAP beeswarm for the tuned XGBoost over training clients. TreeSHAP divides the credit among the collinear `PAY_1`, `MAX_DELAY` and `N_MONTHS_DELAYED`; read them as a block.

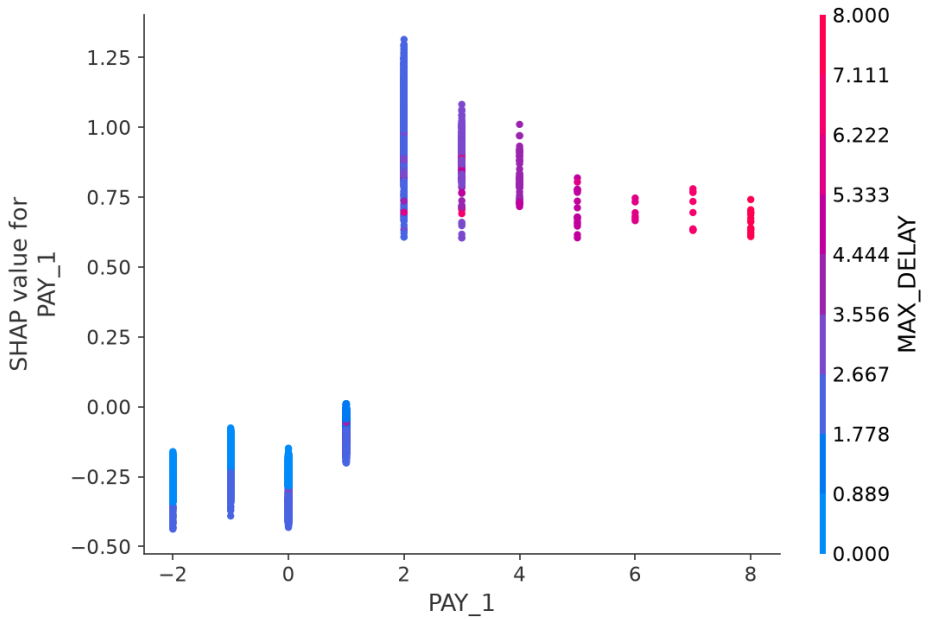


Figure A23: TreeSHAP dependence plot for the most recent repayment status. TreeSHAP divides the credit among the collinear `PAY_1`, `MAX_DELAY` and `N_MONTHS_DELAYED`; read them as a block.

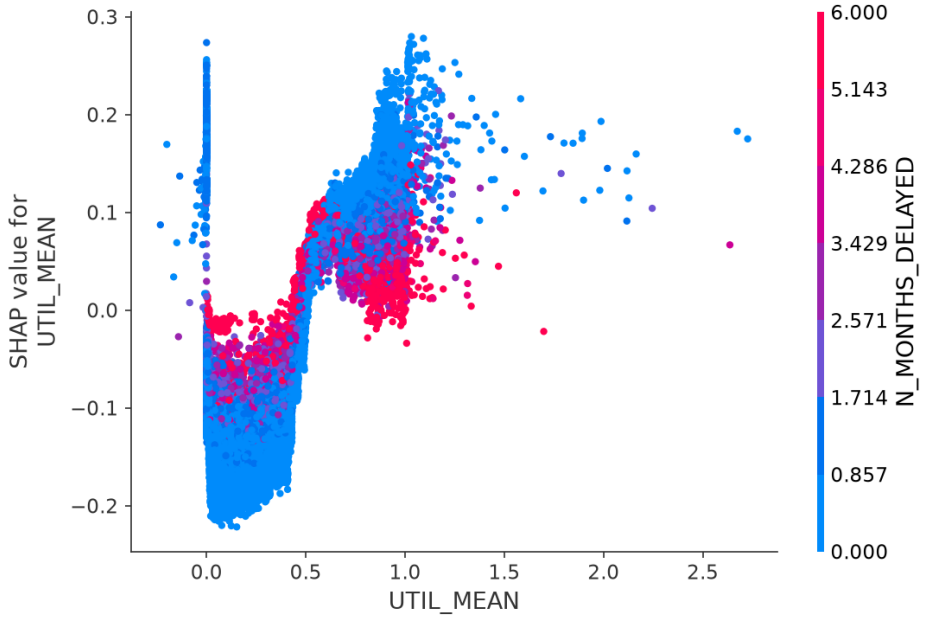


Figure A24: TreeSHAP dependence plot for mean utilisation.

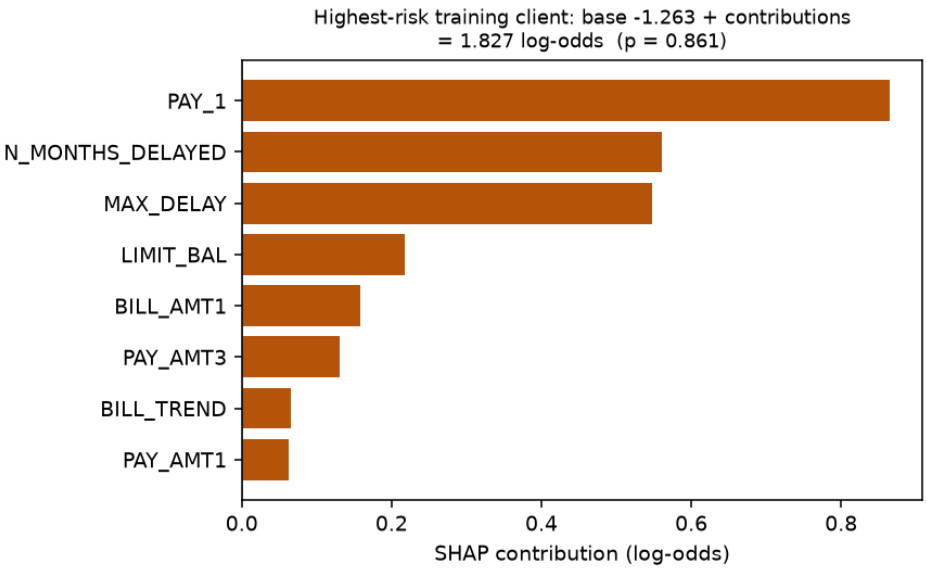


Figure A25: One client decomposed: base value plus each feature's contribution.

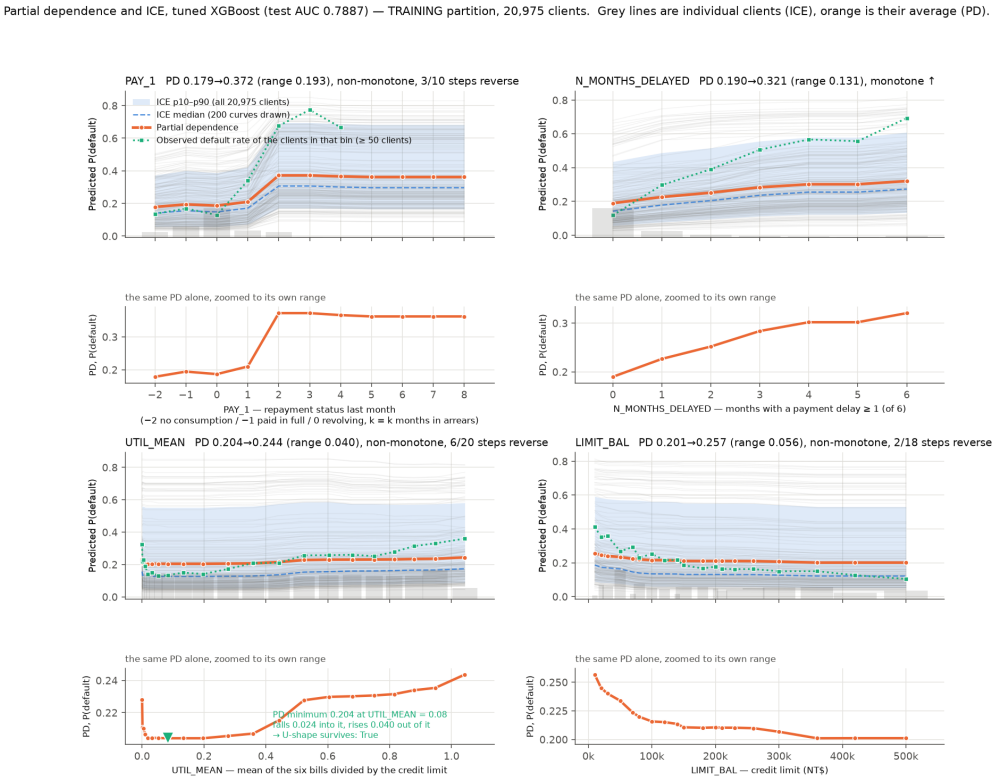


Figure A26: Partial dependence with ICE curves for the four features that matter. The partial dependence holds the derived delay and utilisation features fixed while one column moves, so some swept rows cannot exist in the data. The partial dependence of `PAY_1` rises far less than its raw default rate (Table A21).

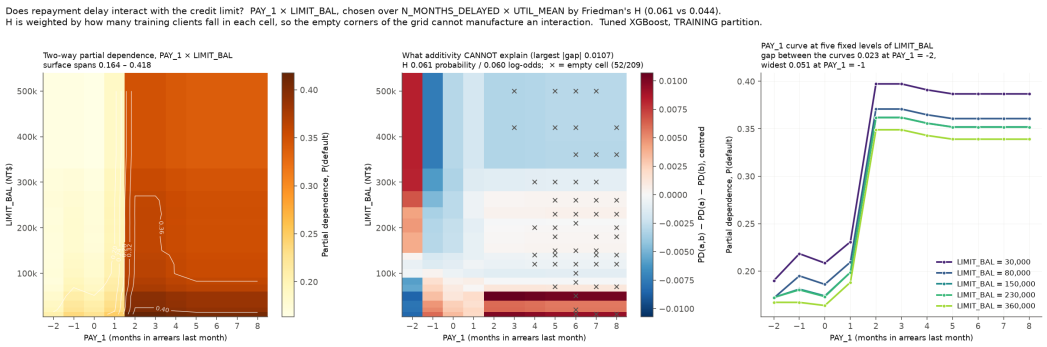


Figure A27: Two-way partial dependence; cells with no training client are crossed (25 % of the PAY_1 × LIMIT_BAL cells contain no training client). The partial dependence holds the derived delay and utilisation features fixed while one column moves, so some swept rows cannot exist in the data.

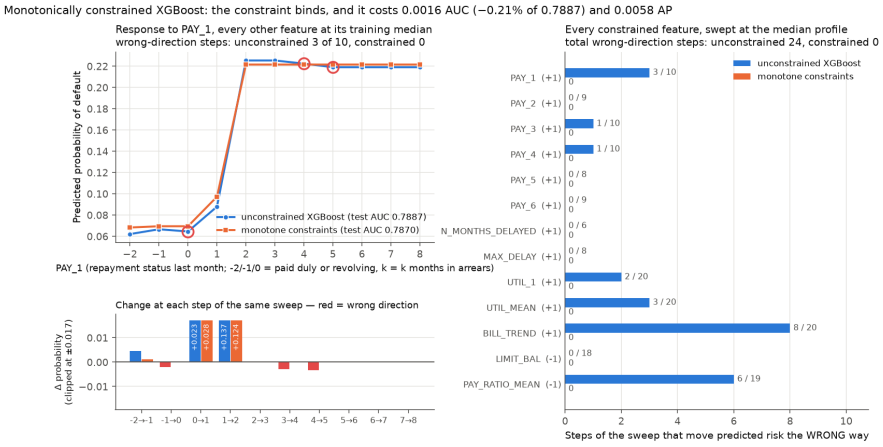


Figure A28: Predicted probability against the most recent repayment status, with and without monotone constraints. The sweep holds MAX_DELAY and N_MONTHS_DELAYED fixed (‘as frozen’); Table A29 gives the consistent sweep.

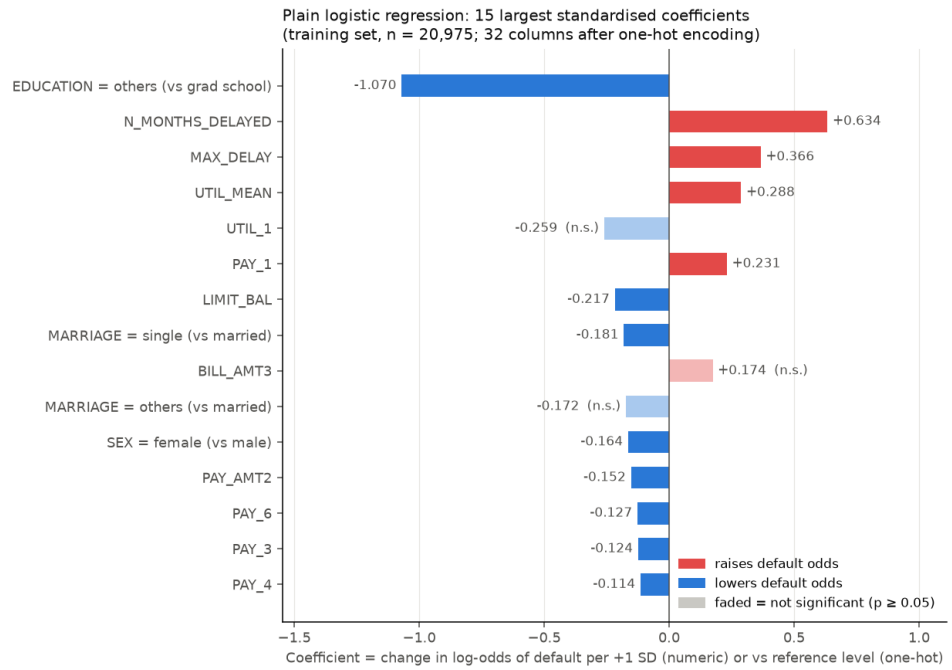


Figure A29: Standardised logistic coefficients: dummy coefficients are per level, numeric ones per standard deviation. Read the bill and utilisation block, and `PAY_2`–`PAY_6` (conditional on the delay counters), as blocks; CV chose almost no shrinkage. The 'others' education coefficient reflects a coding artefact (documented 'others' merged with the undocumented codes 0, 5 and 6), not an education effect.

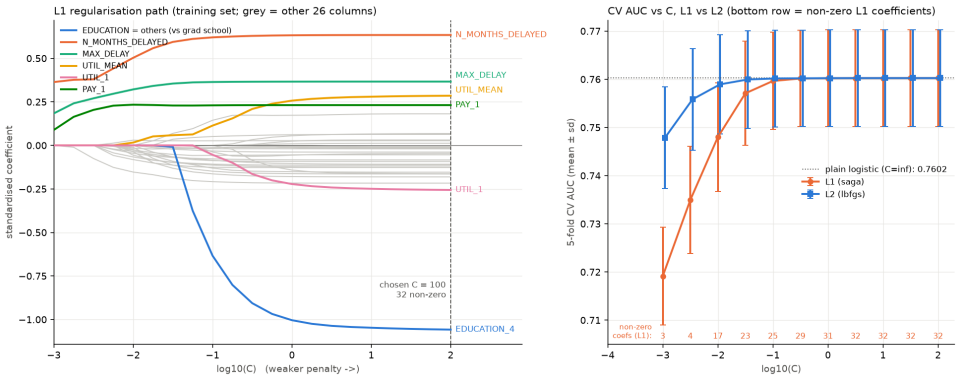


Figure A30: Lasso path: order of entry of the predictors as the penalty is relaxed.

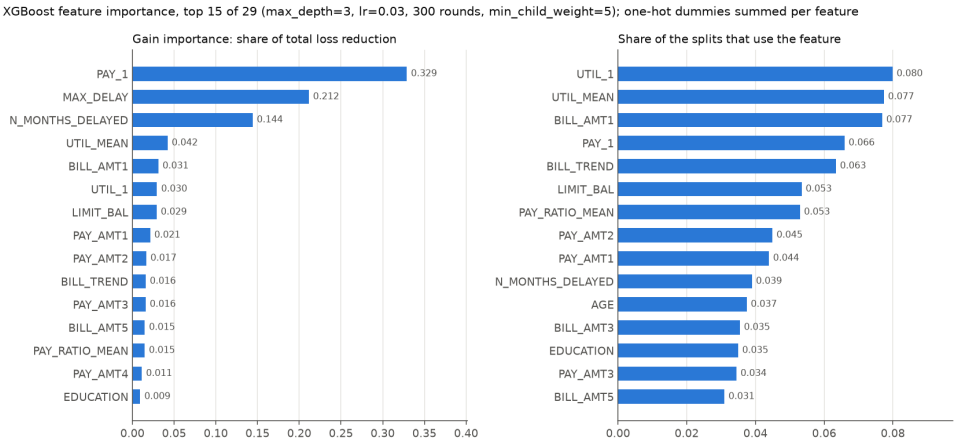


Figure A31: XGBoost gain-based importance.

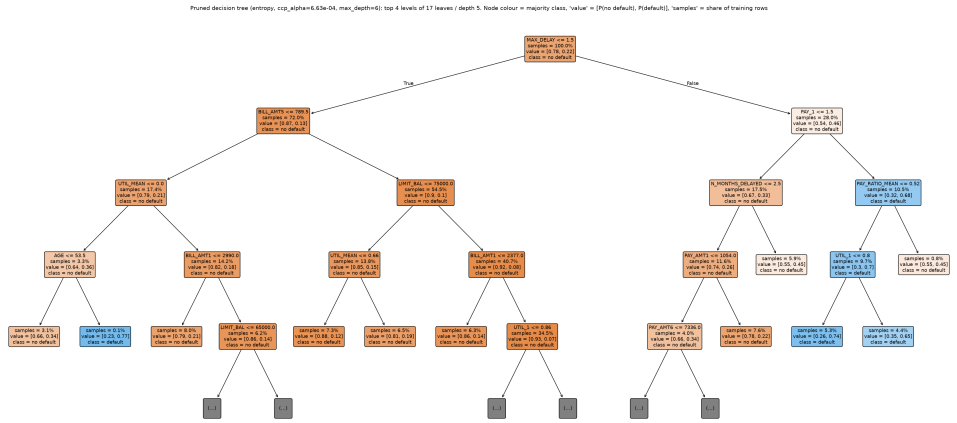


Figure A32: The pruned decision tree, top levels.

A.7 Class imbalance and the leakage demonstration

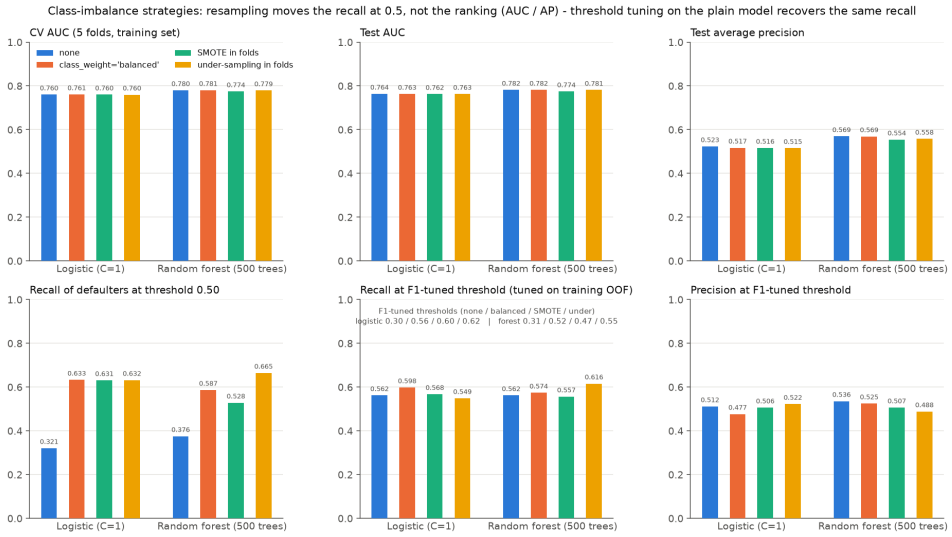


Figure A33: Resampling strategies applied inside the folds: AUC is unchanged, the operating point moves.

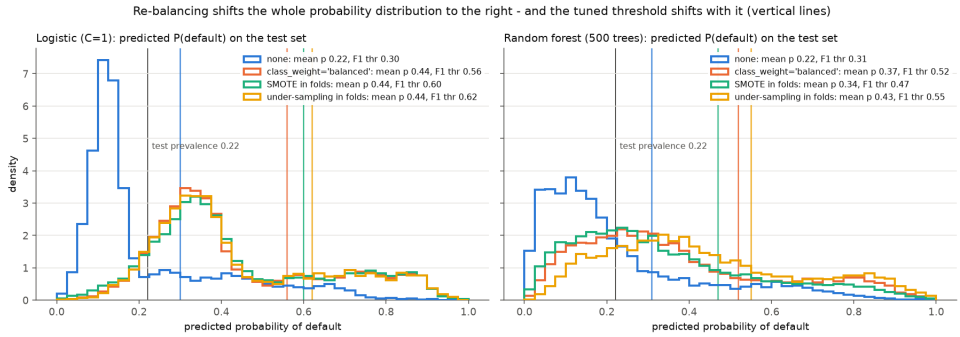


Figure A34: Where each resampling strategy puts the tuned threshold.

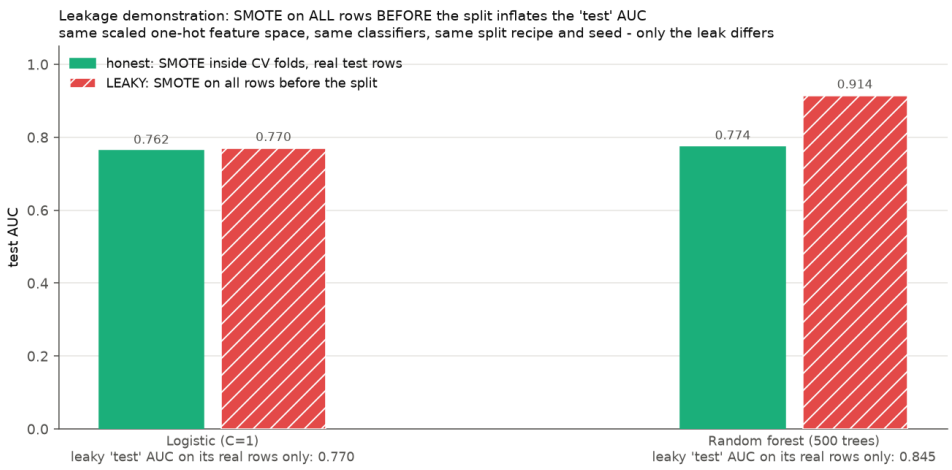


Figure A35: Negative control: SMOTE applied before the split inflates the apparent test AUC.

A.8 Model selection diagnostics

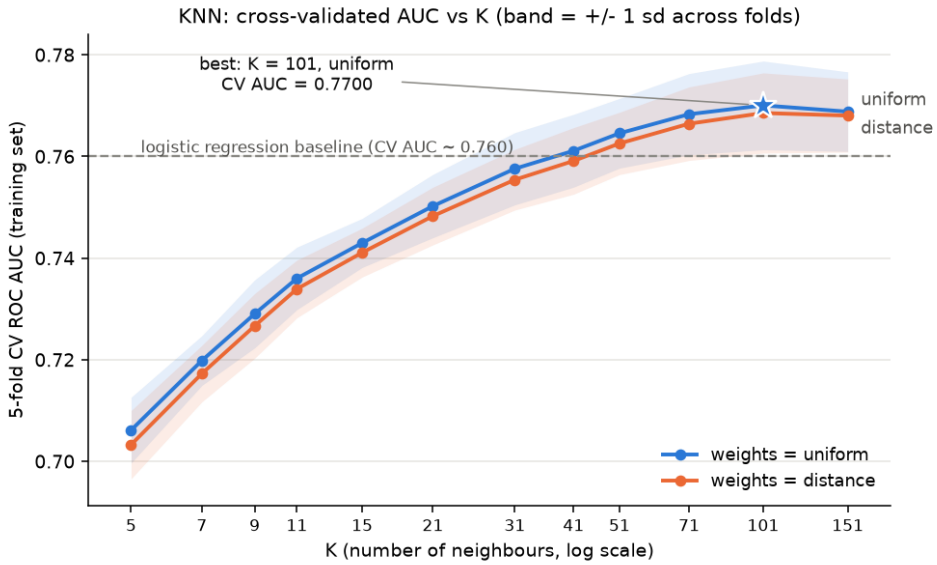


Figure A36: KNN: cross-validated AUC against K.

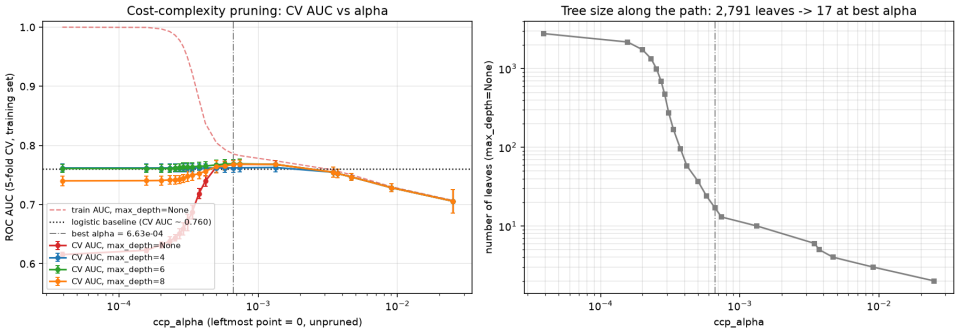


Figure A37: Cost-complexity pruning path.

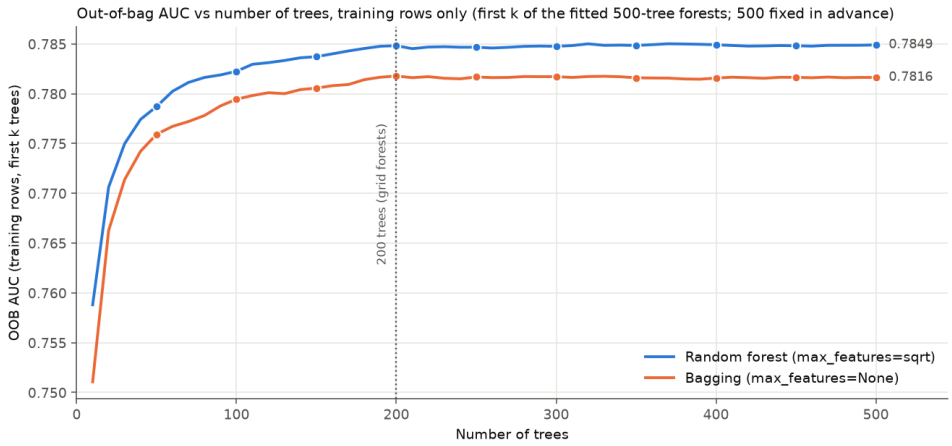


Figure A38: Random forest and bagging: out-of-bag AUC on the training rows against the number of trees (the tree count, 500, was fixed in advance).

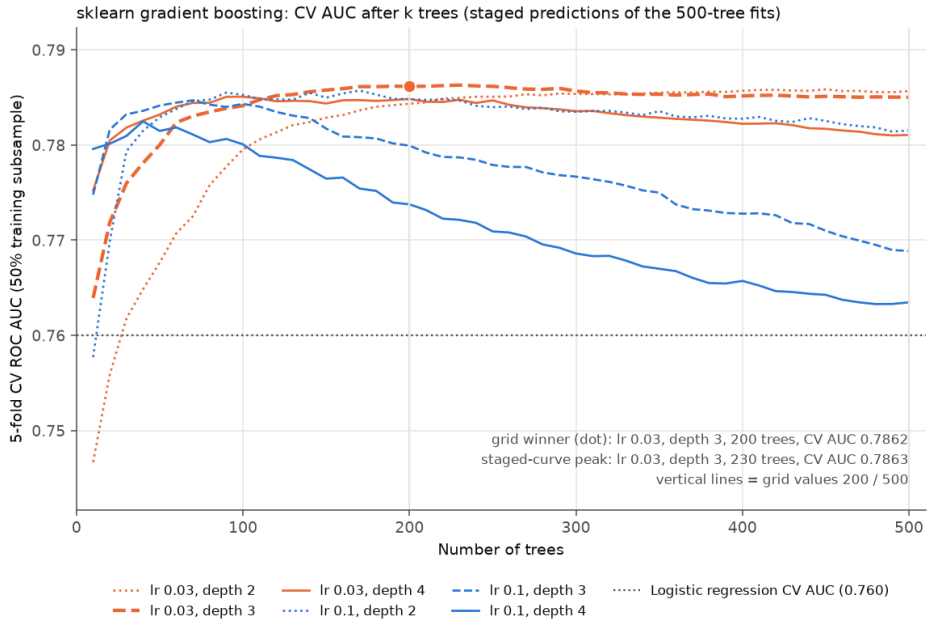


Figure A39: Gradient boosting: cross-validated AUC against the number of rounds.

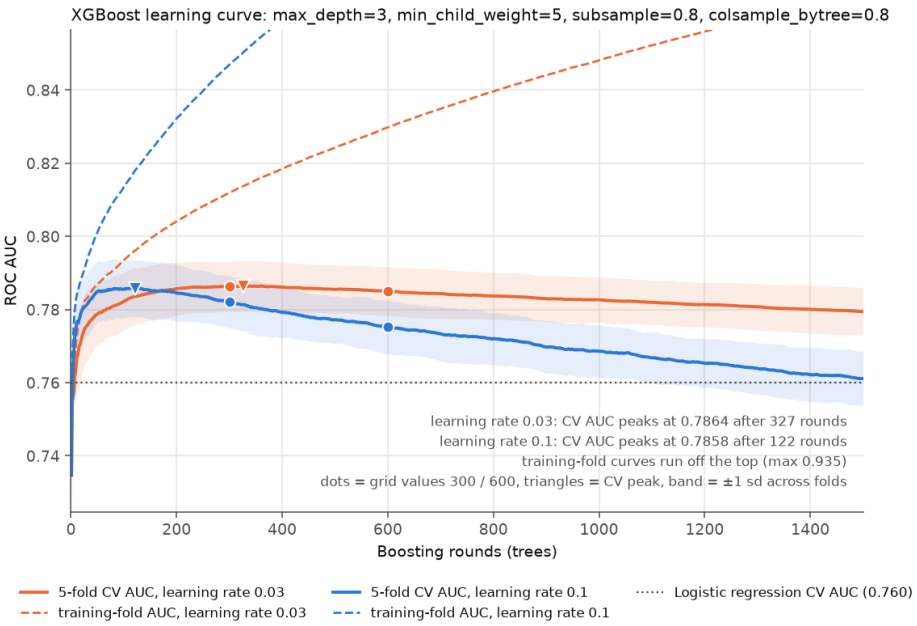


Figure A40: XGBoost learning curves at two learning rates.

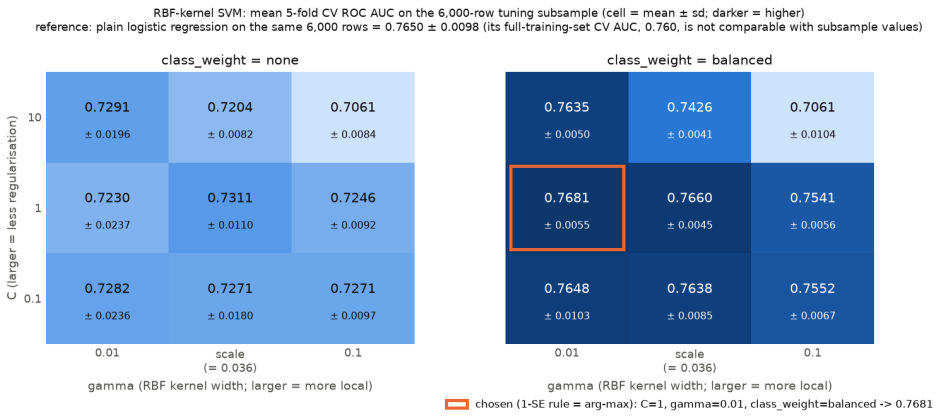


Figure A41: RBF support-vector machine: cross-validated AUC over C and γ.

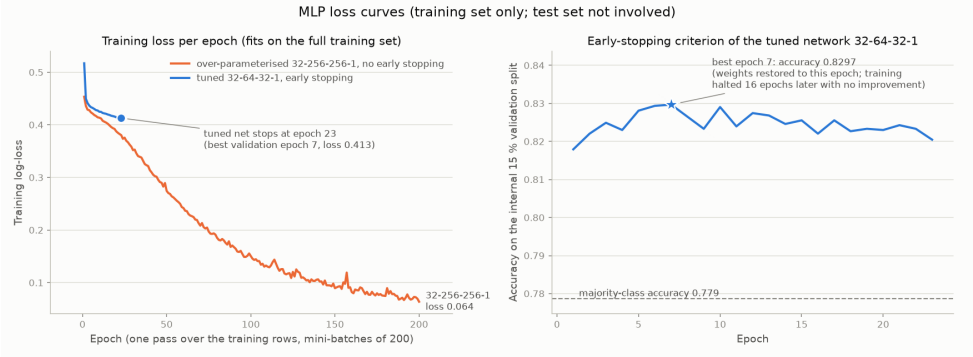


Figure A42: Neural network: training loss and validation AUC by epoch.

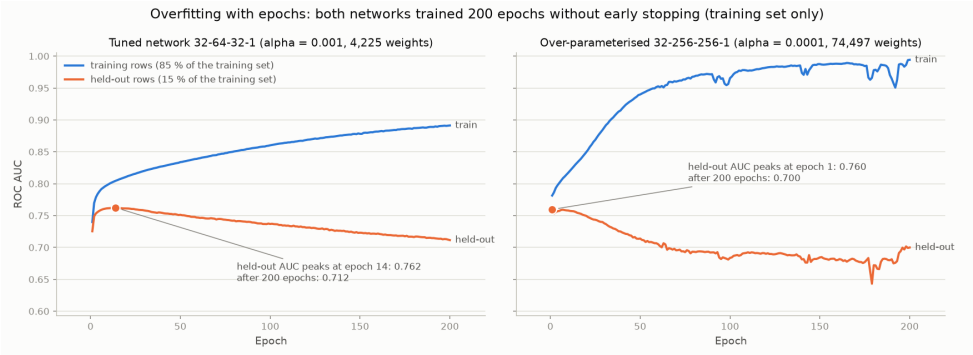


Figure A43: Over-parameterised network: the over-fitting diagnostic.

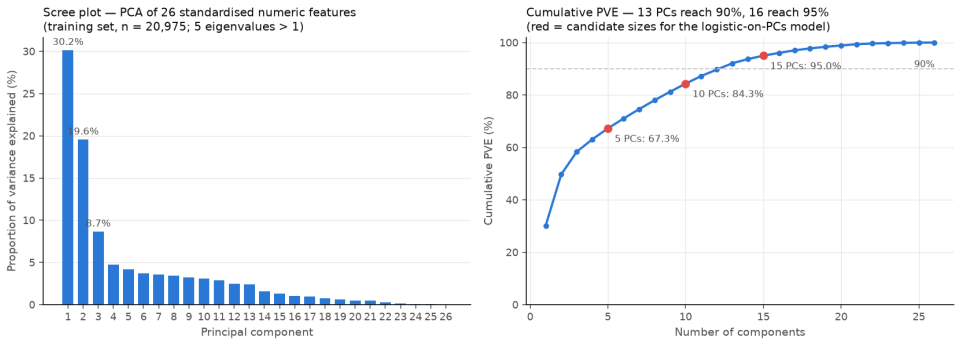


Figure A44: PCA scree plot and cumulative variance explained.

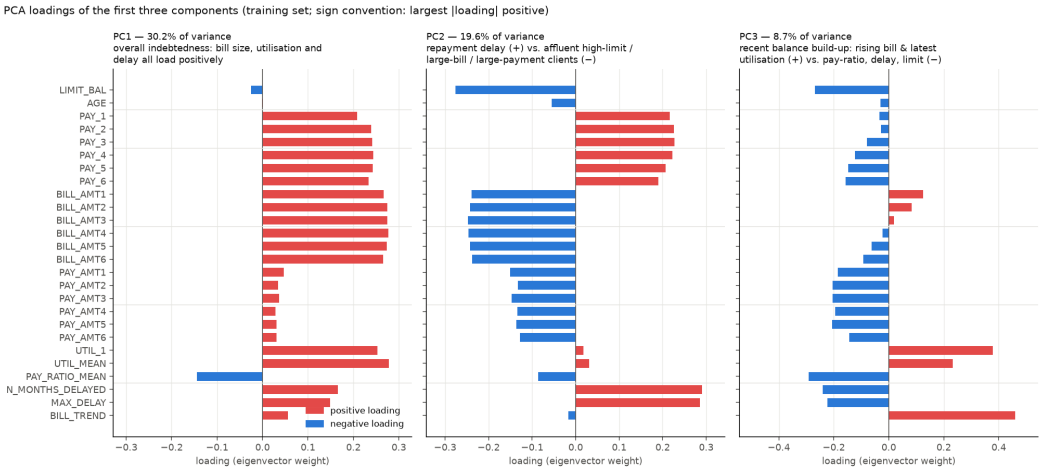


Figure A45: Loadings of the first three principal components.

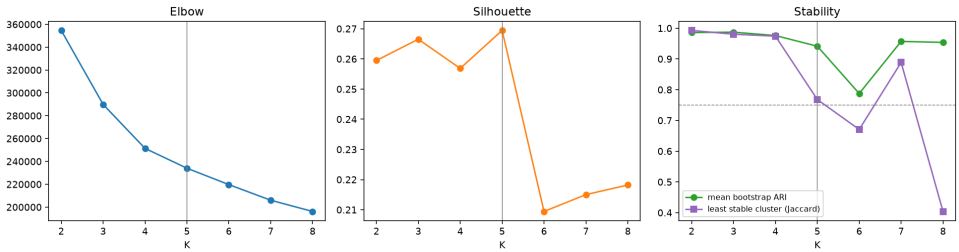


Figure A46: K-means on the first 10 principal components: silhouette, bootstrap ARI and minimum cluster Jaccard against K (Table A17); reproduction on 21 October of RUAN Hangjing's 30 September analysis, training partition only.